

SWISSMICROS DM42

DM42 SOLVER HANDBOOK

From Simple Equations to Advanced Numerical Modelling

A practical long-term reference for formulating, entering, solving, diagnosing and verifying scalar numerical models on the SwissMicros DM42

Research edition 2026

Calculator behavior checked against DM42 User Manual v3.42 dated 24 September 2026, the current DM42 Quick Reference and Functions documentation, the SwissMicros Encoder/Decoder, Free42 documentation and history, and the HP-42S manuals.

How to use this handbook

This is a book about the Solver, not a general owner's manual. Read Parts I and II in order if the Solver is new to you. Then work through enough of Part III that storing known values and selecting an unknown becomes automatic. Later parts can be used by discipline, model type or diagnostic symptom.

KEY IDEA Every example follows the same chain: original equation → choose an unknown → form $f(x)=0$ → implement the residual → supply known values → choose guesses → solve → inspect the returned state → verify the physical and mathematical meaning.

The DM42 Solver is a numerical root finder. It does not rearrange symbols and it does not prove that a returned number is the solution you intended. It repeatedly executes your program at trial values and tries to drive the final X-register result of that program to zero.

Examples use SI units unless another coherent system is stated. Financial rates are identified as decimal or percent. Angle mode is stated whenever trigonometric functions appear. Program names and variable names are kept within the seven-character HP-42S convention for portability.

CAUTION The DM42 uses decimal128 arithmetic, but a model can still be ill-conditioned, discontinuous, wrongly scaled or physically incomplete. More digits do not repair a poor residual.

Keystroke notation

Notation	Meaning	Example
[KEY]	A physical key or named shifted function	[RCL], [SOLVER], [PRGM]
{MENU}	A soft key shown at the bottom of the LCD	{A}, {KEPLER}
{TEXT}	Type the literal alpha name, then press ENTER	{MVAR}, {RHO}
LCD: text	A message or label shown by the calculator	LCD: Sign Reversal
01 instruction	A stored program line	05 RCL× "R"
E	Exponent-entry/display glyph	45E-6

The current DM42 Quick Reference describes operations by function name, for example PRGM and SOLVER. This handbook follows that convention. When a function is printed as a shifted legend on your keyboard overlay, press SHIFT first; when it is in a menu, press the corresponding soft key. In Program-entry mode, enter ordinary instructions with their physical keys or menu soft keys. Pressing [XEQ] begins an actual XEQ instruction; use it only when the stored program must call a labelled program or subroutine.

EXPERT TIP For long listings, the official SwissMicros Encoder/Decoder is faster and less error-prone than hand entry. Early examples nevertheless show a complete key-by-key path so the stored listing can be checked line for line.

Contents

Part I — Understanding the Solver

Part II — Building a Solver Program

Part III — Trivial Examples That Build Fluency

Part IV — Intermediate Examples

Part V — Advanced Examples

Part VI — Advanced Techniques

Part VII — Extreme Case Studies

Appendices A–N — Quick reference, templates and diagnostics

Glossary and bibliography

PART I

Understanding the Solver

What the Solver is actually doing, what its messages mean, and why guesses and verification matter.

1 Numerical solving as residual hunting

An equation states a balance. A numerical root finder needs a single number whose desired value is zero. If the model says left side = right side, define the residual as left side – right side. A solution is a value of the chosen unknown for which the residual vanishes, or is acceptably close to zero.

original equation $\rightarrow$ left side – right side = $f(x) \rightarrow$ seek $f(x) = 0$

Solving an equation is therefore different from evaluating a formula. A one-way formula accepts inputs and returns one output. A Solver model declares several MVAR quantities and can often treat any one of them as x , while the rest remain known parameters. The program itself simply recalls the current named values and leaves the residual in X .

KEY IDEA Now ask what the calculator is actually trying to make equal to zero. If you cannot answer that in one sentence, the residual is not ready to program.

Task	What happens	Typical tool
Evaluate	Known inputs are substituted once	Ordinary RPN calculation
Solve	One named value is varied repeatedly	SOLVER variable menu
Find a zero	The final program result is driven toward 0	Residual program
Change unknown	A different MVAR is varied; the model is unchanged	Same Solver program
Choose a root	Guesses steer the search toward one branch	One or two estimates
Validate	The result is substituted into the original model	Independent check

1.1 The standard Solver program

```

01 LBL "MODEL"
02 MVAR "X"
03 MVAR "A"
04 MVAR "B"
05 ... calculate f(X, A, B) ...
06 END

```

- A global LBL makes the model selectable from SOLVER.
- Contiguous MVAR instructions immediately after the label define the variable menu.
- The body must return one real scalar residual in X .
- END terminates the program and returns the residual to the Solver.
- Named variables are global state. Clear or overwrite stale values deliberately.

CAUTION The Solver cannot find a complex unknown. If evaluation produces a complex value at a trial point, that point is treated as outside the usable real domain.

2 How the iterative search behaves

The Solver evaluates the residual at guesses, compares results, and proposes new trial points. A pair of points with opposite residual signs is powerful because a continuous function must cross zero somewhere between them. The word continuous matters: a pole can also change sign.

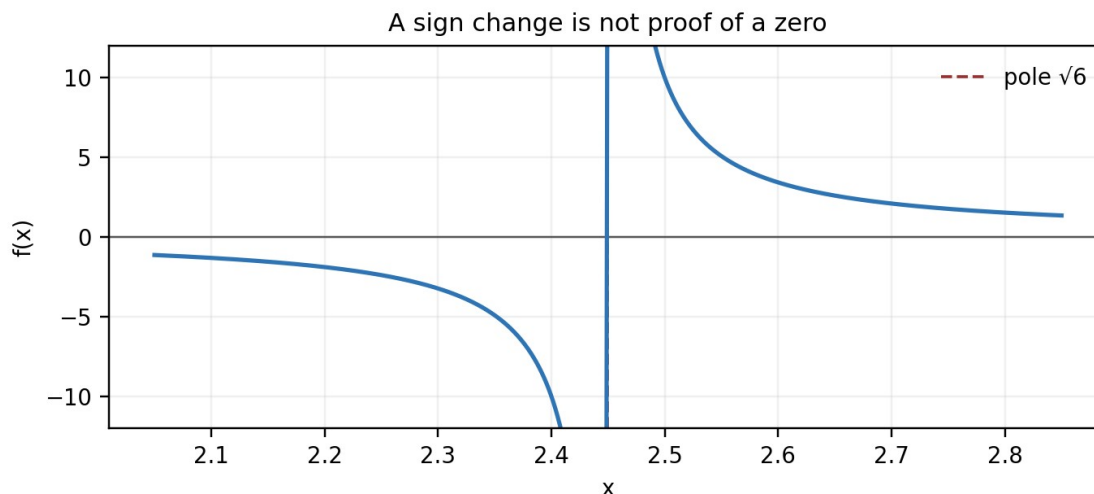


Figure 1 A bracket around a pole narrows toward a discontinuity, not a root.

A single stored guess tells the Solver where to begin. Two successive values stored into the unknown provide two estimates. If they bracket a continuous crossing, convergence is usually faster and more predictable. Two estimates on the same side can still work, but they carry less information.

SOLVER TRAP A sign reversal proves only that neighboring residuals have opposite signs. It does not prove that the original model has a finite zero between them.

2.1 Domains and discontinuities

- Square roots require non-negative arguments when a real residual is expected.
- Logarithms require positive arguments.
- Denominators must stay away from zero.
- Fractional powers of negative numbers may become complex.
- Physical variables may be more restricted than their mathematical domains: mass, concentration and friction factor are normally positive.

A good bracket lies inside both the mathematical and physical domain. If the model contains a known boundary, bracket on one side of it; do not invite the Solver to cross the boundary.

3 The result state in X Y Z and T

On completion the Solver returns more than the displayed candidate. Current Free42 behavior, inherited by the DM42 core, preserves the classic HP-42S result structure and includes later improvements in recognising vertical asymptotes.

Register	Meaning after SOLVE	How to use it
X	Best guess or solution candidate	Report only after verification
Y	Previous guess	Compare with X to see the final neighborhood
Z	Residual $f(X)$ evaluated at the X candidate	Inspect magnitude and units
T	Result code 0 through 4	Branch or diagnose programmatically

T code	Interpretation	Display message or action
0	A solution candidate was found	No warning message; still inspect Z
1	Opposite signs at neighboring points while the residual diverged	Sign Reversal
2	Approximation to an extremum of $ f(x) $	Extremum
3	One or both initial evaluations were outside the domain	Bad Guess(es)
4	Sampled values were indistinguishable	Constant?

VERIFICATION A tiny Z is necessary but not always sufficient. Also substitute X into the original equation, compare the original left and right sides, confirm the units, and decide whether the selected root is physically admissible.

3.1 Why the messages occur

Sign Reversal can indicate a pole. The search has found adjacent representable values with opposite signs, but the residual grows away from zero. This result deserves checking.

Extremum means $|f(x)|$ stopped improving near a local minimum or maximum. A tangent or double root may correctly produce this message because the residual touches zero without crossing it. It can also mean that the closest point is still far from zero.

Bad Guess(es) means a starting evaluation produced a mathematical error or an unusable value. Move both guesses into the domain; do not merely widen the interval.

Constant? means all sampled values were the same at the Solver's working resolution. The function may genuinely be constant, the program may ignore the unknown, or the residual may be so badly scaled that changes are numerically invisible.

4 Initial guesses as part of the solution

Guess selection is not an afterthought. It is where your analytical understanding enters the numerical process. Before touching the calculator, estimate signs, scale, boundaries and the number of plausible roots.

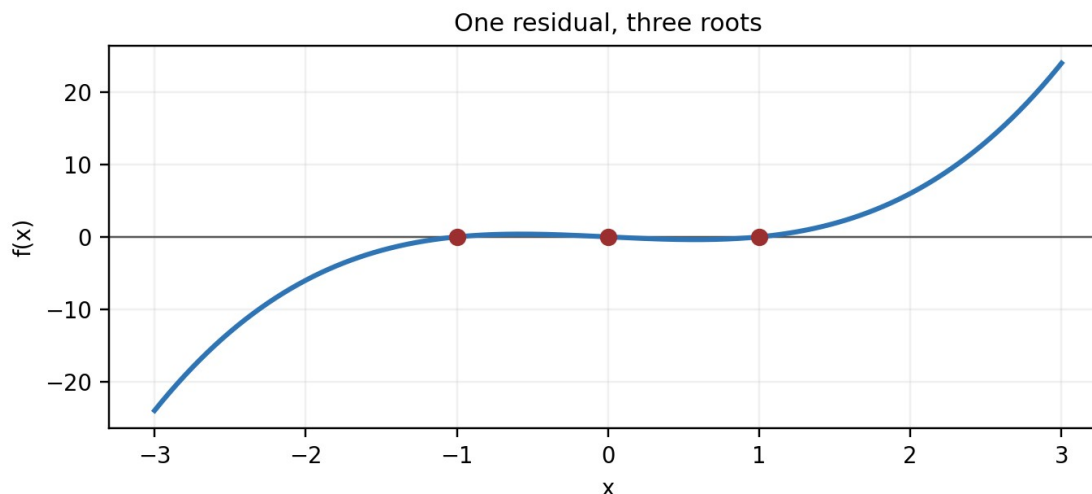


Figure 2 A single residual may have several legitimate roots.

Situation	Guess strategy	Reason
One smooth crossing	Two estimates on opposite sides	Creates a bracket
Several roots	Scan a coarse grid; bracket each sign change separately	Makes root choice explicit
Double or tangent root	Start close to the suspected extremum	There may be no sign change
Pole or domain boundary	Keep both estimates on the same valid side	Prevents false sign reversal
Very wide scale	Solve for a transformed variable such as $u = \ln x$	Improves conditioning
Physical interval	Use engineering bounds	Rejects irrelevant mathematical roots

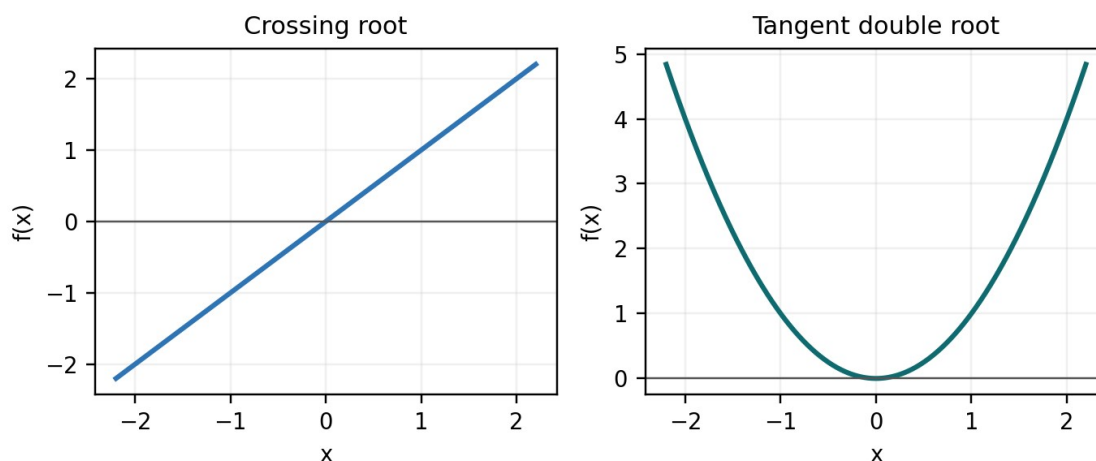


Figure 3 A crossing root changes sign; a repeated root may only touch zero.

5 Precision conditioning scaling and units

The DM42's decimal128 arithmetic offers 34 decimal digits and a very large exponent range, but the Solver still observes the residual through finite arithmetic and the behavior of the model. A poorly conditioned equation can turn tiny input changes into large output changes. Bad scaling can make meaningful changes disappear beside a huge constant.

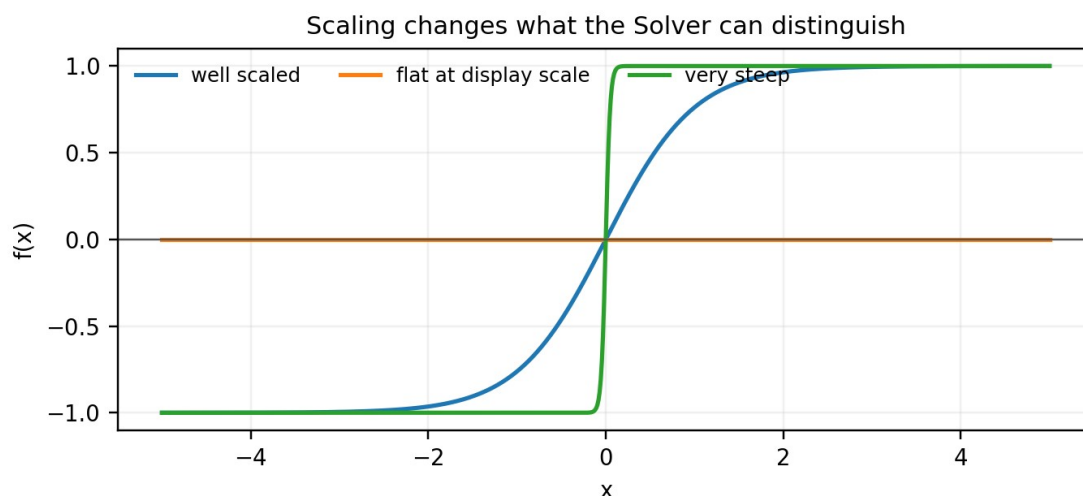


Figure 4 Flat, steep and well-scaled residuals ask different things of the search.

- Prefer dimensionless ratios when practical.
- Scale variables so likely values are not separated by dozens of orders of magnitude.
- Avoid subtracting nearly equal large quantities if an algebraically equivalent stable form exists.
- Use $\text{LN}1+X$ for $\ln(1+x)$ near $x = 0$ and $E^{\uparrow}X-1$ for e^x-1 near $x = 0$ where the current function set supports them.
- Keep all terms of a residual in the same units.
- Write 45 μm as 45E-6 m; do not hide the conversion.

EXPERT TIP A residual may be multiplied by any non-zero constant without changing its roots. Dividing by a representative scale often improves interpretability: a dimensionless residual near 1 is easier to diagnose than a residual near 10^{20} .

PART II

Building a Solver Program

A complete first model from equation to stack plan, program entry, solving, and reuse.

6 The complete workflow with A plus B equals C

We begin with a problem that is almost insultingly easy. That is useful: no algebraic difficulty can hide a keystroke or stack mistake.

1. Write the original equation: $A + B = C$.
2. Choose the unknown. First choose A.
3. Move everything to one side: $A + B - C = 0$.
4. Plan the RPN evaluation: recall A, add B, subtract C.
5. Create a global label and declare A, B and C with MVAR.
6. Enter the residual body and END.
7. Leave Program-entry mode and select the model from SOLVER.
8. Store B and C, store one or two estimates for A, then press {A} with no pending number.
9. Inspect X, Y, Z and T; substitute the result into the original equation.
10. Keep the program and later solve the same model for B or C.

$$A + B = C \rightarrow f(A,B,C) = A + B - C$$

6.1 Stack plan

Instruction	T	Z	Y	X
RCL A	—	—	—	A
RCL+ B	—	—	—	A + B
RCL- C	—	—	—	A + B - C

RCL+ and RCL- are compact and leave no temporary values above the residual. The final X value is exactly what the Solver needs.

6.2 Program and every entry action

```

01 LBL "SUM"
02 MVAR "A"
03 MVAR "B"
04 MVAR "C"
05 RCL "A"
06 RCL+ "B"
07 RCL- "C"
08 END

```

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {SUM} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {A} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {B} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {C} [ENTER]

Line 05: [RCL] {A} [ENTER]

Line 06: [RCL] [+] {B} [ENTER]

Line 07: [RCL] [-] {C} [ENTER]

The new space already ends in .END.; finish: [PRGM]

CAUTION If an alpha name already exists, the DM42 may offer it as a menu choice. Selecting that menu item and typing the same name are equivalent only if the spelling is identical.

6.3 Solve for A

Use $B = 12$ and $C = 20$. Estimates 0 and 20 bracket the root.

[SOLVER] {SUM} → 12 {B} → 20 {C} → 0 {A} → 20 {A} → {A}

VERIFICATION $X = 8$. Evaluate the original balance: $8 + 12 = 20$. Inspect Z; it should be zero at the available numerical precision. $T = 0$ identifies a solution candidate.

6.4 Solve the same model for C

Store $A = 8$ and $B = 12$. The residual program is unchanged; only the selected unknown changes.

8 {A} → 12 {B} → 0 {C} → 30 {C} → {C}

WHY THIS WORKS MVAR does not make the program symbolic. The Solver temporarily writes trial values into the selected named variable while the same numerical residual reads all three variables.

7 A repeatable entry and checking routine

Before entry	During entry	Before solving	After solving
Short label; variables ≤ 7 characters; units defined; residual written	Compare every stored line with the listing; test the residual manually at a known point	Set angle mode; store every known variable; choose domain-safe guesses	Inspect X/Y/Z/T; substitute; reject invalid roots; record units and significant figures

KEY IDEA Program entry is not complete when the last line appears. It is complete when the stored listing, the intended RPN evaluation, and an independent numerical check all agree.

PART III

Trivial Examples That Build Fluency

The arithmetic is easy so the Solver workflow can become automatic. Every program includes a complete entry sequence.

8 Why solve the obvious

Ordinary arithmetic is faster for most problems in this part. The examples exist to build reliable habits: define variables, form the residual, enter the program, store known values, distinguish storing a guess from selecting the unknown, and verify the result.

KEY IDEA For each example, say the residual aloud before pressing SOLVE. The habit scales better than memorising keystrokes.

8.1 A first zero x plus 3 equals 7

This deliberately tiny problem separates the algebra from the Solver ritual. The answer is obvious; that is the point. You can concentrate on what the calculator is doing.

Variable	Meaning	Unit
X	unknown number	1

$$X + 3 = 7$$

Residual: $f(X) = X + 3 - 7$

Calculator mode: Any angle mode

Program listing

```
01 LBL "XPLUS3"
02 MVAR "X"
03 RCL "X"
04 3
05 +
06 7
07 -
08 END
```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {XPLUS3} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {X} [ENTER]

Line 03: [RCL] {X} [ENTER]

Line 04: [3]

Line 05: [+]

Line 06: [7]

Line 07: [-]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: none. Unknown: X.

[SOLVER] {XPLUS3} → 0 {X} → 10 {X} → {X}

VERIFICATION Expected result: $X = 4$. Substitution gives $4 + 3 - 7 = 0$ exactly.

What can go wrong

- Pressing {X} after typing a number stores a guess; pressing {X} with no pending number starts the solve.

WHY THIS WORKS The program does not solve algebraically. It evaluates a residual repeatedly until the residual is zero.

8.2 One reusable model A plus B equals C

This is the standard skeleton. All three quantities are menu variables, so any one can be the unknown while the other two are held fixed.

Variable	Meaning	Unit
A	first addend	1
B	second addend	1
C	sum	1

$$A + B = C$$

Residual: $f = A + B - C$

Calculator mode: Any angle mode

Program listing

```

01 LBL "SUM"
02 MVAR "A"
03 MVAR "B"
04 MVAR "C"
05 RCL "A"
06 RCL+ "B"
07 RCL- "C"
08 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {SUM} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {A} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {B} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {C} [ENTER]

Line 05: [RCL] {A} [ENTER]

Line 06: [RCL] [+] {B} [ENTER]

Line 07: [RCL] [-] {C} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $B = 12$; $C = 20$. Unknown: A .

[SOLVER] {SUM} → 12 {B} → 20 {C} → 0 {A} → 20 {A} → {A}

VERIFICATION Expected result: $A = 8$. $8 + 12 - 20 = 0$.

Reuse the model: store $A = 8$ and $B = 12$, then press {C}; the Solver returns $C = 20$.

WHY THIS WORKS MVAR turns a one-way formula into a small model. The same residual can solve for A , B , or C .

8.3 Force mass and acceleration

Use Newton's second law as a three-variable Solver model.

Variable	Meaning	Unit
F	force	N
M	mass	kg
A	acceleration	m/s ²

$$F = M \times A$$

Residual: $f = M \times A - F$

Calculator mode: Any angle mode

Program listing

```

01 LBL "FORCE"
02 MVAR "F"
03 MVAR "M"
04 MVAR "A"
05 RCL "M"
06 RCL× "A"
07 RCL- "F"
08 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {FORCE} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {F} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {M} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {A} [ENTER]

Line 05: [RCL] {M} [ENTER]

Line 06: [RCL] [×] {A} [ENTER]

Line 07: [RCL] [-] {F} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $M = 15$; $A = 9.81$. Unknown: F .

[SOLVER] {FORCE} → 15 {M} → 9.81 {A} → 100 {F} → 200 {F} → {F}

VERIFICATION Expected result: $F = 147.15$ N. $15 \times 9.81 - 147.15 = 0$.

Reuse the model: with $F = 147.15$ and $A = 9.81$, solve for M to recover 15 kg.

WHY THIS WORKS A dimensional check predicts kilograms times metres per second squared, which is a newton.

8.4 Distance at constant speed

A constant-speed trip is simple arithmetic, but it is excellent rehearsal for named values and units.

Variable	Meaning	Unit
D	distance	m
V	speed	m/s
T	time	s

$$D = V \times T$$

Residual: $f = V \times T - D$

Calculator mode: Any angle mode

Program listing

```

01 LBL "MOTION"
02 MVAR "D"
03 MVAR "V"
04 MVAR "T"
05 RCL "V"
06 RCL× "T"
07 RCL- "D"
08 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {MOTION} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {D} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {V} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {T} [ENTER]

Line 05: [RCL] {V} [ENTER]

Line 06: [RCL] [×] {T} [ENTER]

Line 07: [RCL] [-] {D} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $V = 25$; $T = 18$. Unknown: D .

[SOLVER] {MOTION} → 25 {V} → 18 {T} → 0 {D} → 1000 {D} → {D}

VERIFICATION Expected result: $D = 450$ m. $25 \times 18 - 450 = 0$.

Solve the same model for T with $D = 450$ and $V = 25$.

WHY THIS WORKS Known values are stored once; the Solver program reads them on every evaluation.

8.5 Ohm's law

Build a bidirectional V-I-R model for a resistor.

Variable	Meaning	Unit
V	voltage	V
I	current	A

Variable	Meaning	Unit
R	resistance	Ω

$$V = I \times R$$

Residual: $f = I \times R - V$

Calculator mode: Any angle mode

Program listing

```

01 LBL "OHMS"
02 MVAR "V"
03 MVAR "I"
04 MVAR "R"
05 RCL "I"
06 RCL× "R"
07 RCL- "V"
08 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {OHMS} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {V} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {I} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {R} [ENTER]

Line 05: [RCL] {I} [ENTER]

Line 06: [RCL] [×] {R} [ENTER]

Line 07: [RCL] [-] {V} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $I = 0.02$; $R = 600$. Unknown: V .

[SOLVER] {OHMS} → 0.02 {I} → 600 {R} → 0 {V} → 20 {V} → {V}

VERIFICATION Expected result: $V = 12$ V. $0.02 \times 600 - 12 = 0$.

With $V = 12$ and $I = 0.02$, solve for R to obtain 600 Ω .

WHY THIS WORKS SI units keep the residual dimensionally consistent: amperes times ohms gives volts.

8.6 Electrical power

Relate power, voltage and current without building a separate formula for every unknown.

Variable	Meaning	Unit
P	power	W
V	voltage	V
I	current	A

$$P = V \times I$$

Residual: $f = V \times I - P$

Calculator mode: Any angle mode

Program listing

```
01 LBL "POWER"
02 MVAR "P"
03 MVAR "V"
04 MVAR "I"
05 RCL "V"
06 RCL× "I"
07 RCL- "P"
08 END
```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {POWER} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {P} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {V} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {I} [ENTER]

Line 05: [RCL] {V} [ENTER]

Line 06: [RCL] [×] {I} [ENTER]

Line 07: [RCL] [-] {P} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $V = 12$; $I = 0.75$. Unknown: P .

[SOLVER] {POWER} → 12 {V} → 0.75 {I} → 0 {P} → 20 {P} → {P}

VERIFICATION Expected result: $P = 9$ W. $12 \times 0.75 - 9 = 0$.

WHY THIS WORKS A Solver model earns its keep when the unknown changes from one use to the next.

8.7 Mass density and volume

Find the mass of an aluminium component from density and volume.

Variable	Meaning	Unit
M	mass	kg
RHO	density	kg/m ³
VOL	volume	m ³

$$M = \rho \times \text{VOL}$$

Residual: $f = \text{RHO} \times \text{VOL} - M$

Calculator mode: Any angle mode

Program listing

```
01 LBL "DENSITY"
02 MVAR "M"
03 MVAR "RHO"
04 MVAR "VOL"
05 RCL "RHO"
```

```

06 RCL× "VOL"
07 RCL- "M"
08 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {DENSITY} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {M} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {RHO} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {VOL} [ENTER]

Line 05: [RCL] {RHO} [ENTER]

Line 06: [RCL] [×] {VOL} [ENTER]

Line 07: [RCL] [-] {M} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: RHO = 2700; VOL = 0.001. Unknown: M.

[SOLVER] {DENSIT} → 2700 {RHO} → 0.001 {VOL} → 0 {M} → 10 {M} → {M}

VERIFICATION Expected result: M = 2.7 kg. $2700 \times 0.001 - 2.7 = 0$.

WHY THIS WORKS The variable name RHO is an ASCII-friendly label; use ρ in the surrounding mathematics, not as an unverified variable name.

8.8 Sensible heat

Calculate the heat added to water when its temperature rises by 5 K.

Variable	Meaning	Unit
Q	heat	J
M	mass	kg
C	specific heat	J/(kg·K)
DT	temperature rise	K

$$Q = M \times C \times \Delta T$$

Residual: $f = M \times C \times DT - Q$

Calculator mode: Any angle mode

Program listing

```

01 LBL "HEAT"
02 MVAR "Q"
03 MVAR "M"
04 MVAR "C"
05 MVAR "DT"
06 RCL "M"
07 RCL× "C"
08 RCL× "DT"
09 RCL- "Q"
10 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {HEAT} [ENTER]
 Line 02: [PGM.FCN] [▲] {MVAR} {Q} [ENTER]
 Line 03: [PGM.FCN] [▲] {MVAR} {M} [ENTER]
 Line 04: [PGM.FCN] [▲] {MVAR} {C} [ENTER]
 Line 05: [PGM.FCN] [▲] {MVAR} {DT} [ENTER]
 Line 06: [RCL] {M} [ENTER]
 Line 07: [RCL] [×] {C} [ENTER]
 Line 08: [RCL] [×] {DT} [ENTER]
 Line 09: [RCL] [-] {Q} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $M = 2$; $C = 4184$; $DT = 5$. Unknown: Q .

[SOLVER] {HEAT} $\rightarrow 2 \{M\} \rightarrow 4184 \{C\} \rightarrow 5 \{DT\} \rightarrow 0 \{Q\} \rightarrow 50000 \{Q\} \rightarrow \{Q\}$

VERIFICATION Expected result: $Q = 41\,840\text{ J}$. $2 \times 4184 \times 5 - 41\,840 = 0$.

WHY THIS WORKS Named variables let the residual read almost like the dimensional equation.

8.9 Ideal gas pressure

Solve the ideal-gas law in SI units for pressure.

Variable	Meaning	Unit
P	pressure	Pa
V	volume	m ³
N	amount	mol
R	gas constant	J/(mol·K)
T	temperature	K

$$P \times V = N \times R \times T$$

Residual: $f = P \times V - N \times R \times T$

Calculator mode: Any angle mode

Program listing

```
01 LBL "IDEALG"
02 MVAR "P"
03 MVAR "V"
04 MVAR "N"
05 MVAR "R"
06 MVAR "T"
07 RCL "P"
08 RCL× "V"
09 RCL "N"
10 RCL× "R"
11 RCL× "T"
12 -
```

13 END

Program entry**Start:** [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {IDEALG} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {P} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {V} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {N} [ENTER]

Line 05: [PGM.FCN] [▲] {MVAR} {R} [ENTER]

Line 06: [PGM.FCN] [▲] {MVAR} {T} [ENTER]

Line 07: [RCL] {P} [ENTER]

Line 08: [RCL] [×] {V} [ENTER]

Line 09: [RCL] {N} [ENTER]

Line 10: [RCL] [×] {R} [ENTER]

Line 11: [RCL] [×] {T} [ENTER]

Line 12: [-]

The new space already ends in .END.; finish: [PRGM]**Solve**Known data: $V = 0.024$; $N = 1$; $R = 8.314462618$; $T = 300$. Unknown: P .

[SOLVER] {IDEALG} → 0.024 {V} → 1 {N} → 8.314462618 {R} → 300 {T} → 50000 {P} → 200000 {P} → {P}

VERIFICATION Expected result: $P = 103\,930.7827$ Pa. $P \times 0.024$ and $1 \times 8.314462618 \times 300$ both equal 2494.3387854 J.**WHY THIS WORKS** The two sides have the same units. A residual that subtracts unlike dimensions is never a valid physical model.**8.10 Percentage increase**

Find the final price after a percentage increase, then reuse the model to recover the rate.

Variable	Meaning	Unit
BASE	starting value	currency
RATE	increase	%
FINAL	ending value	currency

$$\text{FINAL} = \text{BASE} \times (1 + \text{RATE} \div 100)$$

Residual: $f = \text{BASE} \times (1 + \text{RATE} \div 100) - \text{FINAL}$ **Calculator mode:** Any angle mode**Program listing**

```

01 LBL "PCTINC"
02 MVAR "BASE"
03 MVAR "RATE"
04 MVAR "FINAL"
05 1
06 RCL "RATE"
07 100
08 ÷

```

```

09  +
10  RCL× "BASE"
11  RCL- "FINAL"
12  END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {PCTINC} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {BASE} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {RATE} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {FINAL} [ENTER]
Line 05: [1]
Line 06: [RCL] {RATE} [ENTER]
Line 07: [1] [0] [0]
Line 08: [÷]
Line 09: [+]
Line 10: [RCL] [×] {BASE} [ENTER]
Line 11: [RCL] [-] {FINAL} [ENTER]

```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: BASE = 80; RATE = 12.5. Unknown: FINAL.

[SOLVER] {PCTINC} → 80 {BASE} → 12.5 {RATE} → 80 {FINAL} → 100 {FINAL} → {FINAL}

VERIFICATION Expected result: FINAL = 90. $80 \times 1.125 - 90 = 0$.

Store BASE = 80 and FINAL = 90, then solve RATE to recover 12.5%.

WHY THIS WORKS Percent is not a unitless fraction until divided by 100.

8.11 Simple interest

Use a decimal annual rate and solve the simple-interest relation.

Variable	Meaning	Unit
A	accumulated amount	currency
P	principal	currency
R	rate per year	1/year
T	time	year

$$A = P \times (1 + R \times T)$$

Residual: $f = P \times (1 + R \times T) - A$

Calculator mode: Any angle mode

Program listing

```

01  LBL "SIMPINT"
02  MVAR "A"
03  MVAR "P"
04  MVAR "R"
05  MVAR "T"

```

```

06 RCL "R"
07 RCL× "T"
08 1
09 +
10 RCL× "P"
11 RCL- "A"
12 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {SIMPINT} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {A} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {P} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {R} [ENTER]

Line 05: [PGM.FCN] [▲] {MVAR} {T} [ENTER]

Line 06: [RCL] {R} [ENTER]

Line 07: [RCL] [×] {T} [ENTER]

Line 08: [1]

Line 09: [+]

Line 10: [RCL] [×] {P} [ENTER]

Line 11: [RCL] [-] {A} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: P = 10000; R = 0.08; T = 1.5. Unknown: A.

[SOLVER] {SIMPIN} → 10000 {P} → 0.08 {R} → 1.5 {T} → 10000 {A} → 12000 {A} → {A}

VERIFICATION Expected result: A = 11 200. $10\,000 \times (1 + 0.08 \times 1.5) = 11\,200$.

WHY THIS WORKS State whether a financial rate is entered as 8 or 0.08. Here R is explicitly a decimal rate.

8.12 Break even quantity

Find how many units cover a fixed cost when each unit contributes price minus variable cost.

Variable	Meaning	Unit
Q	quantity	unit
FIX	fixed cost	currency
PRICE	price per unit	currency/unit
VC	variable cost	currency/unit

$$\text{PRICE} \times Q = \text{FIX} + \text{VC} \times Q$$

Residual: $f = (\text{PRICE} - \text{VC}) \times Q - \text{FIX}$

Calculator mode: Any angle mode

Program listing

```

01 LBL "BREAKEY"
02 MVAR "Q"
03 MVAR "FIX"

```

```

04 MVAR "PRICE"
05 MVAR "VC"
06 RCL "PRICE"
07 RCL- "VC"
08 RCL× "Q"
09 RCL- "FIX"
10 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {BREAKEV} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {Q} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {FIX} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {PRICE} [ENTER]

Line 05: [PGM.FCN] [▲] {MVAR} {VC} [ENTER]

Line 06: [RCL] {PRICE} [ENTER]

Line 07: [RCL] [-] {VC} [ENTER]

Line 08: [RCL] [×] {Q} [ENTER]

Line 09: [RCL] [-] {FIX} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $\text{FIX} = 24000$; $\text{PRICE} = 80$; $\text{VC} = 50$. Unknown: Q .

[SOLVER] {BREAKE} $\rightarrow 24000$ {FIX} $\rightarrow 80$ {PRICE} $\rightarrow 50$ {VC} $\rightarrow 0$ {Q} $\rightarrow 2000$ {Q} $\rightarrow \{Q\}$

VERIFICATION Expected result: $Q = 800$ units. $(80 - 50) \times 800 - 24\,000 = 0$.

WHY THIS WORKS Before reaching for SOLVE, inspect the contribution margin. If $\text{PRICE} \leq \text{VC}$, no positive break-even quantity exists.

PART IV

Intermediate Examples

Nonlinear equations, domains, multiple roots and models where numerical solving begins to save real effort.

9 From rearrangement to genuine root finding

The examples now include powers, exponentials, logarithmic structure and multiple branches. Analytical inspection remains the first step. The Solver is strongest when the equation is clear but explicit rearrangement is inconvenient or the unknown changes from one use to the next.

9.1 Projectile range and two launch angles

For level launch and landing with no air resistance, find the angle that gives a 70 m range at 30 m/s. The model has two physical roots.

Variable	Meaning	Unit
RNG	horizontal range	m
V	launch speed	m/s
ANG	launch angle	degree
G	gravity	m/s ²

$$\text{RNG} = V^2 \times \sin(2 \times \text{ANG}) \div G$$

Residual: $f = V^2 \sin(2 \text{ ANG}) \div G - \text{RNG}$

Calculator mode: DEG

Program listing

```

01 LBL "PROJ"
02 MVAR "RNG"
03 MVAR "V"
04 MVAR "ANG"
05 MVAR "G"
06 RCL "V"
07 X↑2
08 RCL "ANG"
09 2
10 ×
11 SIN
12 ×
13 RCL÷ "G"
14 RCL- "RNG"
15 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {PROJ} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {RNG} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {V} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {ANG} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {G} [ENTER]
Line 06: [RCL] {V} [ENTER]
Line 07: [X↑2]
Line 08: [RCL] {ANG} [ENTER]
Line 09: [2]
Line 10: [×]

```

Line 11: [SIN]
 Line 12: [×]
 Line 13: [RCL] [÷] (G) [ENTER]
 Line 14: [RCL] [-] (RNG) [ENTER]
 The new space already ends in .END.; finish: [PRGM]

Solve

Known data: RNG = 70; V = 30; G = 9.80665. Unknown: ANG.

[SOLVER] {PROJ} → 70 {RNG} → 30 {V} → 9.80665 {G} → 10 {ANG} → 40 {ANG} → {ANG}

VERIFICATION Expected result: ANG = 24.85315005°. Substitution returns 70.00000000 m; the residual is below 10^{-12} m in the independent check.

Reuse the model with guesses 50 and 80 to obtain ANG = 65.14684995°.

What can go wrong

- RAD mode silently changes the model.
- A bracket spanning both roots can steer unpredictably.
- The no-drag model is an approximation.

WHY THIS WORKS The guesses select the low-angle branch. A second bracket from 50° to 80° selects the high-angle root.

9.2 Quadratic drag terminal speed

Balance weight against quadratic drag for an 80 kg body.

Variable	Meaning	Unit
VT	terminal speed	m/s
M	mass	kg
G	gravity	m/s ²
RHO	air density	kg/m ³
CD	drag coefficient	1
AREA	area	m ²

$$\frac{1}{2} \text{RHO} \times \text{CD} \times \text{AREA} \times \text{VT}^2 = \text{M} \times \text{G}$$

Residual: $f = \frac{1}{2} \text{RHO} \text{CD} \text{AREA} \text{VT}^2 - \text{M} \text{G}$

Calculator mode: Any angle mode

Program listing

```
01 LBL "TERMVEL"
02 MVAR "VT"
03 MVAR "M"
04 MVAR "G"
05 MVAR "RHO"
06 MVAR "CD"
07 MVAR "AREA"
08 .5
09 RCL× "RHO"
```

```

10 RCL× "CD"
11 RCL× "AREA"
12 RCL "VT"
13 X+2
14 ×
15 RCL "M"
16 RCL× "G"
17 −
18 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {TERMVEL} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {VT} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {M} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {G} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {RHO} [ENTER]
Line 06: [PGM.FCN] [▲] {MVAR} {CD} [ENTER]
Line 07: [PGM.FCN] [▲] {MVAR} {AREA} [ENTER]
Line 08: [.] [5]
Line 09: [RCL] [×] {RHO} [ENTER]
Line 10: [RCL] [×] {CD} [ENTER]
Line 11: [RCL] [×] {AREA} [ENTER]
Line 12: [RCL] {VT} [ENTER]
Line 13: [X+2]
Line 14: [×]
Line 15: [RCL] {M} [ENTER]
Line 16: [RCL] [×] {G} [ENTER]
Line 17: [−]

```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $M = 80$; $G = 9.80665$; $RHO = 1.225$; $CD = 1$; $AREA = 0.7$. Unknown: VT .

```

[SOLVER] {TERMVE} → 80 {M} → 9.80665 {G} → 1.225 {RHO} → 1 {CD} → 0.7 {AREA} → 0
{VT} → 100 {VT} → {VT}

```

VERIFICATION Expected result: $VT = 42.77630471$ m/s. Drag and weight each evaluate to about 784.532 N.

What can go wrong

- Using weight M instead of mass M changes units.
- The coefficient and area depend on posture and flow regime.

WHY THIS WORKS A positive bracket is a physical constraint. The algebra also has a negative root, but negative speed is not the intended terminal-speed magnitude.

9.3 RC charging time

Find when a capacitor reaches 3 V while charging toward 5 V through 10 k Ω and 100 μ F.

Variable	Meaning	Unit
V	capacitor voltage	V
VS	supply voltage	V

Variable	Meaning	Unit
T	time	s
R	resistance	Ω
C	capacitance	F

$$V = VS \times (1 - e^{(-T \div (R \times C))})$$

Residual: $f = VS(1 - e^{(-T/(RC))}) - V$

Calculator mode: Any angle mode

Program listing

```

01 LBL "RCCHG"
02 MVAR "V"
03 MVAR "VS"
04 MVAR "T"
05 MVAR "R"
06 MVAR "C"
07 RCL "T"
08 +/-
09 RCL "R"
10 RCL× "C"
11 ÷
12 E↑X
13 +/-
14 1
15 +
16 RCL× "VS"
17 RCL- "V"
18 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {RCCHG} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {V} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {VS} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {T} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {R} [ENTER]
Line 06: [PGM.FCN] [▲] {MVAR} {C} [ENTER]
Line 07: [RCL] {T} [ENTER]
Line 08: [+/-]
Line 09: [RCL] {R} [ENTER]
Line 10: [RCL] [×] {C} [ENTER]
Line 11: [÷]
Line 12: [E↑X]
Line 13: [+/-]
Line 14: [1]
Line 15: [+]
Line 16: [RCL] [×] {VS} [ENTER]
Line 17: [RCL] [-] {V} [ENTER]

```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $V = 3$; $V_S = 5$; $R = 10000$; $C = 100\text{E-}6$. Unknown: T .

[SOLVER] {RCCHG} → 3 {V} → 5 {VS} → 10000 {R} → 100E-6 {C} → 0 {T} → 5 {T} → {T}

VERIFICATION Expected result: $T = 0.9162907319 \text{ s}$. $5(1 - \exp(-T/1)) = 3.000000000 \text{ V}$.

What can go wrong

- Entering 100 instead of $100\text{E-}6$ for C is a million-fold unit error.
- $V \geq V_S$ has no finite solution in the ideal model.

WHY THIS WORKS The domain requires $0 \leq V < V_S$ for a finite positive charging time.

9.4 Diode and resistor operating point

Find the diode voltage in a 5 V source, 1 k Ω resistor and Shockley diode model.

Variable	Meaning	Unit
VD	diode voltage	V
VS	supply	V
R	resistance	Ω
IS	saturation current	A
N	ideality factor	1
VT	thermal voltage	V

$$IS(e^{(VD/(N VT))} - 1) = (VS - VD) \div R$$

Residual: $f = \text{diode current} - \text{resistor current}$

Calculator mode: Any angle mode

Program listing

```

01 LBL "DIODE"
02 MVAR "VD"
03 MVAR "VS"
04 MVAR "R"
05 MVAR "IS"
06 MVAR "N"
07 MVAR "VT"
08 RCL "VD"
09 RCL "N"
10 RCL× "VT"
11 ÷
12 E↑X
13 1
14 −
15 RCL× "IS"
16 RCL "VS"
17 RCL − "VD"
18 RCL÷ "R"
19 −
20 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```
Line 01: [PGM.FCN] {LBL} {DIODE} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {VD} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {VS} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {R} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {IS} [ENTER]
Line 06: [PGM.FCN] [▲] {MVAR} {N} [ENTER]
Line 07: [PGM.FCN] [▲] {MVAR} {VT} [ENTER]
Line 08: [RCL] {VD} [ENTER]
Line 09: [RCL] {N} [ENTER]
Line 10: [RCL] [×] {VT} [ENTER]
Line 11: [÷]
Line 12: [E+X]
Line 13: [1]
Line 14: [-]
Line 15: [RCL] [×] {IS} [ENTER]
Line 16: [RCL] {VS} [ENTER]
Line 17: [RCL] [-] {VD} [ENTER]
Line 18: [RCL] [÷] {R} [ENTER]
Line 19: [-]
```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: VS = 5; R = 1000; IS = 1E-12; N = 1.8; VT = 0.02585. Unknown: VD.

```
[SOLVER] {DIODE} → 5 {VS} → 1000 {R} → 1E-12 {IS} → 1.8 {N} → 0.02585 {VT} → 0.5
{VD} → 1.2 {VD} → {VD}
```

VERIFICATION Expected result: VD = 1.028425990 V. Both currents are approximately 3.971574 mA.

What can go wrong

- The exponential can overflow for absurdly large guesses.
- VT depends on temperature; the ideality model is approximate.

WHY THIS WORKS The operating point is the intersection of two device relations. This is exactly the kind of scalar implicit problem the Solver handles well.

9.5 Weak acid equilibrium

Estimate the hydrogen-ion concentration of 0.1 mol/L acetic acid using $K_a = 1.8 \times 10^{-5}$ and the simple monoprotic equilibrium model.

Variable	Meaning	Unit
H	[H ⁺]	mol/L
C	formal acid concentration	mol/L
KA	acid constant	mol/L

$$K_A = H^2 \div (C - H)$$

Residual: $f = H^2/(C - H) - K_A$

Calculator mode: Any angle mode

Program listing

```
01 LBL "WEAKAC"
02 MVAR "H"
03 MVAR "C"
04 MVAR "KA"
05 RCL "H"
06 X↑2
07 RCL "C"
08 RCL- "H"
09 ÷
10 RCL- "KA"
11 END
```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {WEAKAC} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {H} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {C} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {KA} [ENTER]

Line 05: [RCL] {H} [ENTER]

Line 06: [X↑2]

Line 07: [RCL] {C} [ENTER]

Line 08: [RCL] [-] {H} [ENTER]

Line 09: [÷]

Line 10: [RCL] [-] {KA} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $C = 0.1$; $K_A = 1.8 \times 10^{-5}$. Unknown: H .

[SOLVER] {WEAKAC} → 0.1 {C} → 1.8×10^{-5} {KA} → 1×10^{-6} {H} → 0.01 {H} → {H}

VERIFICATION Expected result: $H = 0.001332670973$ mol/L; pH = 2.875277. Substitution reproduces K_a to better than 10^{-14} relative in the independent calculation.

What can go wrong

- This model neglects activities and water autoionisation.
- Concentration units must be consistent.

WHY THIS WORKS The physical domain is $0 < H < C$. A guess at C creates a pole, not chemistry.

9.6 Logistic population time

Find when a logistic population reaches 800 when $K = 1000$, $A = 9$ and $r = 0.35$ per time unit.

Variable	Meaning	Unit
P	population	individual
K	carrying capacity	individual
A	initial-condition parameter	1

Variable	Meaning	Unit
R	growth rate	1/time
T	time	time

$$P = K \div (1 + A e^{(-R T)})$$

Residual: $f = K/(1 + A e^{(-RT)}) - P$

Calculator mode: Any angle mode

Program listing

```

01 LBL "LOGIST"
02 MVAR "P"
03 MVAR "K"
04 MVAR "A"
05 MVAR "R"
06 MVAR "T"
07 RCL "R"
08 RCL× "T"
09 +/-
10 E↑X
11 RCL× "A"
12 1
13 +
14 RCL "K"
15 X<>Y
16 ÷
17 RCL- "P"
18 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {LOGIST} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {P} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {K} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {A} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {R} [ENTER]
Line 06: [PGM.FCN] [▲] {MVAR} {T} [ENTER]
Line 07: [RCL] {R} [ENTER]
Line 08: [RCL] [×] {T} [ENTER]
Line 09: [+/-]
Line 10: [E↑X]
Line 11: [RCL] [×] {A} [ENTER]
Line 12: [1]
Line 13: [+]
Line 14: [RCL] {K} [ENTER]
Line 15: [X<>Y]
Line 16: [÷]
Line 17: [RCL] [-] {P} [ENTER]

```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $P = 800$; $K = 1000$; $A = 9$; $R = 0.35$. Unknown: T .

[SOLVER] {LOGIST} → 800 {P} → 1000 {K} → 9 {A} → 0.35 {R} → 0 {T} → 30 {T} → {T}

VERIFICATION Expected result: $T = 10.23862554$. The evaluated population is 800.0000000.

What can go wrong

- A target $P \geq K$ has no finite positive solution.
- Parameter A depends on how time zero was defined.

WHY THIS WORKS A bounded model supplies analytical guidance: P must lie between 0 and K for the chosen positive parameters.

9.7 Michaelis Menten substrate concentration

Find substrate concentration S that produces rate 8 when $V_{MAX} = 12$ and $K_M = 3$.

Variable	Meaning	Unit
V	reaction rate	rate
VMAX	maximum rate	rate
S	substrate concentration	conc.
KM	Michaelis constant	conc.

$$V = V_{MAX} \times S \div (K_M + S)$$

Residual: $f = V_{MAX} S / (K_M + S) - V$

Calculator mode: Any angle mode

Program listing

```

01 LBL "MICHAEL"
02 MVAR "V"
03 MVAR "VMAX"
04 MVAR "S"
05 MVAR "KM"
06 RCL "VMAX"
07 RCL× "S"
08 RCL "KM"
09 RCL+ "S"
10 ÷
11 RCL- "V"
12 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {MICHAEL} [ENTER]
Line 02: [PGM.FCN] {MVAR} {V} [ENTER]
Line 03: [PGM.FCN] {MVAR} {VMAX} [ENTER]
Line 04: [PGM.FCN] {MVAR} {S} [ENTER]
Line 05: [PGM.FCN] {MVAR} {KM} [ENTER]
Line 06: [RCL] {VMAX} [ENTER]
Line 07: [RCL] [×] {S} [ENTER]
Line 08: [RCL] {KM} [ENTER]
Line 09: [RCL] [+] {S} [ENTER]
Line 10: [÷]
Line 11: [RCL] [-] {V} [ENTER]

```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $V = 8$; $V_{MAX} = 12$; $KM = 3$. Unknown: S .

[SOLVER] {MICHAEL} → 8 {V} → 12 {VMAX} → 3 {KM} → 0 {S} → 30 {S} → {S}

VERIFICATION Expected result: $S = 6$. $12 \times 6 \div (3 + 6) - 8 = 0$.

WHY THIS WORKS The asymptote $V = V_{MAX}$ is a model boundary. Asking for a greater rate is not a numerical challenge; it is impossible in this model.

9.8 Compound growth rate

Find the annual rate that grows 10 000 to 16 000 in six years.

Variable	Meaning	Unit
FV	future value	currency
PV	present value	currency
R	decimal rate	1/year
N	periods	year

$$FV = PV \times (1 + R)^N$$

Residual: $f = PV(1+R)^N - FV$

Calculator mode: Any angle mode

Program listing

```

01 LBL "COMPOUN"
02 MVAR "FV"
03 MVAR "PV"
04 MVAR "R"
05 MVAR "N"
06 1
07 RCL+ "R"
08 RCL "N"
09 Y↑X
10 RCL× "PV"
11 RCL- "FV"
12 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {COMPOUN} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {FV} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {PV} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {R} [ENTER]

Line 05: [PGM.FCN] [▲] {MVAR} {N} [ENTER]

Line 06: [1]

Line 07: [RCL] [+] {R} [ENTER]

Line 08: [RCL] {N} [ENTER]

Line 09: [Y↑X]

Line 10: [RCL] [×] {PV} [ENTER]

Line 11: [RCL] [-] (FV) [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: FV = 16000; PV = 10000; N = 6. Unknown: R.

[SOLVER] {COMPOU} → 16000 {FV} → 10000 {PV} → 6 {N} → 0 {R} → 0.2 {R} → {R}

VERIFICATION Expected result: R = 0.0814837471, or 8.148375% per year. $PV(1+R)^6$ evaluates to 16 000 within rounding.

WHY THIS WORKS A finance model should state whether R is decimal or percent and whether compounding is annual, monthly or continuous.

9.9 Loan interest rate

Infer the monthly rate on a 250 000 loan with 360 equal payments of 1 800, paid at period end.

Variable	Meaning	Unit
PV	loan principal	currency
PMT	payment	currency/month
I	monthly decimal rate	1/month
N	number of payments	month
FAC	internal factor	1

$$PMT = PV I(1+I)^N \div ((1+I)^N - 1)$$

Residual: f = calculated payment – PMT

Calculator mode: Any angle mode

Program listing

```

01 LBL "LOANIR"
02 MVAR "PV"
03 MVAR "PMT"
04 MVAR "I"
05 MVAR "N"
06 RCL "I"
07 1
08 +
09 RCL "N"
10 Y↑X
11 STO "FAC"
12 RCL "I"
13 RCL× "PV"
14 RCL× "FAC"
15 RCL "FAC"
16 1
17 -
18 ÷
19 RCL - "PMT"
20 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} <LOANIR> [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} <PV> [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} <PMT> [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} <I> [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} <N> [ENTER]
Line 06: [RCL] <I> [ENTER]
Line 07: [1]
Line 08: [+ ]
Line 09: [RCL] <N> [ENTER]
Line 10: [Y+X]
Line 11: [ST0] <FAC> [ENTER]
Line 12: [RCL] <I> [ENTER]
Line 13: [RCL] [×] <PV> [ENTER]
Line 14: [RCL] [×] <FAC> [ENTER]
Line 15: [RCL] <FAC> [ENTER]
Line 16: [1]
Line 17: [-]
Line 18: [÷]
Line 19: [RCL] [-] <PMT> [ENTER]
The new space already ends in .END.; finish: [PRGM]

```

Solve

Known data: PV = 250000; PMT = 1800; N = 360. Unknown: I.

[SOLVER] {LOANIR} → 250000 {PV} → 1800 {PMT} → 360 {N} → 0.001 {I} → 0.02 {I} → {I}

VERIFICATION Expected result: I = 0.0065015588 per month; nominal APR = 7.801871%. Recomputed payment is 1 800.000000 within rounding.

What can go wrong

- The formula is for end-of-period payments.
- Nominal APR and effective annual rate are different quantities.

WHY THIS WORKS The helper variable FAC is recalculated on every Solver evaluation. It is not an MVAR because the reader should not edit it.

9.10 Quantal response equilibrium

Solve a symmetric scalar fixed point $p = \text{logistic}(\lambda(2p-1))$ with $\lambda = 1.5$. This is a modelling example, not a universal game-theory law.

Variable	Meaning	Unit
P	probability of action 1	1
LAM	response sensitivity	1

$$P = 1 \div (1 + e^{(-\text{LAM}(2P-1))})$$

Residual: $f = \text{logistic response} - P$

Calculator mode: Any angle mode

Program listing

```

01 LBL "QRE"
02 MVAR "P"
03 MVAR "LAM"
04 RCL "P"
05 2
06 ×
07 1
08 −
09 RCL× "LAM"
10 +/-
11 E↑X
12 1
13 +
14 1
15 X<>Y
16 ÷
17 RCL- "P"
18 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {QRE} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {P} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {LAM} [ENTER]

Line 04: [RCL] {P} [ENTER]

Line 05: [2]

Line 06: [×]

Line 07: [1]

Line 08: [−]

Line 09: [RCL] [×] {LAM} [ENTER]

Line 10: [+/-]

Line 11: [E↑X]

Line 12: [1]

Line 13: [+]

Line 14: [1]

Line 15: [X<>Y]

Line 16: [÷]

Line 17: [RCL] [−] {P} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: LAM = 1.5. Unknown: P.

[SOLVER] {QRE} → 1.5 {LAM} → 0.2 {P} → 0.8 {P} → {P}

VERIFICATION Expected result: P = 0.5. The response at p = 0.5 is exactly 0.5.

What can go wrong

- The payoff normalization is embedded in $2P-1$.
- A mathematical equilibrium need not be empirically descriptive.

WHY THIS WORKS A fixed point becomes a root by subtracting the trial value. Larger λ may create multiple roots, so guess selection becomes part of the model analysis.

PART V

Advanced Applications

Implicit scientific, engineering and financial equations with explicit root ranges, failure modes and independent checks.

10 Serious scalar models

Each model in this part is implicit or transcendental. The program is still conceptually simple: read named variables and return one residual. The hard work lies in defining a defensible model, respecting its domain and interpreting the root.

CAUTION A highly precise root of an approximate model is still only as reliable as the model and its input parameters.

10.1 Kepler's equation

Solve $M = E - e \sin(E)$ for eccentric anomaly E with mean anomaly $M = 0.7$ rad and eccentricity $e = 0.6$.

Variable	Meaning	Unit
M	mean anomaly	rad
E	eccentric anomaly	rad
ECC	orbital eccentricity	1

$$M = E - ECC \times \sin(E)$$

Residual: $f = E - ECC \sin(E) - M$

Calculator mode: RAD

Program listing

```

01 LBL "KEPLER"
02 MVAR "M"
03 MVAR "E"
04 MVAR "ECC"
05 RCL "E"
06 RCL "E"
07 SIN
08 RCL× "ECC"
09 −
10 RCL− "M"
11 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {KEPLER} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {M} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {E} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {ECC} [ENTER]

Line 05: [RCL] {E} [ENTER]

Line 06: [RCL] {E} [ENTER]

Line 07: [SIN]

Line 08: [RCL] [×] {ECC} [ENTER]

Line 09: [−]

Line 10: [RCL] [−] {M} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $M = 0.7$; $ECC = 0.6$. Unknown: E .

[SOLVER] {KEPLER} → 0.7 {M} → 0.6 {ECC} → 0.7 {E} → 1.7 {E} → {E}

VERIFICATION Expected result: $E = 1.273717386613$ rad. $E - 0.6 \sin(E) = 0.700000000000$ to displayed precision.

What can go wrong

- DEG mode produces a numerically plausible but wrong result.
- Reduce M consistently if using a principal 0 to 2π interval.

WHY THIS WORKS For an elliptical orbit $0 \leq e < 1$ and the derivative $1 - e \cos E$ stays positive, so the root is unique for a given M .

Model note: Kepler's equation is a standard orbital relation; use radians for this program.

10.2 Van der Waals molar volume

For one mole of CO_2 -like gas parameters, solve the van der Waals equation at 50 bar and 300 K for molar volume.

Variable	Meaning	Unit
P	pressure	bar
VM	molar volume	L/mol
A	van der Waals a	$\text{L}^2 \cdot \text{bar} / \text{mol}^2$
B	van der Waals b	L/mol
R	gas constant	$\text{L} \cdot \text{bar} / (\text{mol} \cdot \text{K})$
T	temperature	K

$$(P + A/VM^2)(VM - B) = R T$$

Residual: $f = (P + A/VM^2)(VM - B) - RT$

Calculator mode: Any angle mode

Program listing

```

01 LBL "VDW"
02 MVAR "P"
03 MVAR "VM"
04 MVAR "A"
05 MVAR "B"
06 MVAR "R"
07 MVAR "T"
08 RCL "A"
09 RCL "VM"
10 X↑2
11 ÷
12 RCL+ "P"
13 RCL "VM"
14 RCL- "B"
15 ×
16 RCL "R"
17 RCL× "T"

```

```
18 -
19 END
```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```
Line 01: [PGM.FCN] {LBL} {VDW} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {P} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {VM} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {A} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {B} [ENTER]
Line 06: [PGM.FCN] [▲] {MVAR} {R} [ENTER]
Line 07: [PGM.FCN] [▲] {MVAR} {T} [ENTER]
Line 08: [RCL] {A} [ENTER]
Line 09: [RCL] {VM} [ENTER]
Line 10: [X+2]
Line 11: [÷]
Line 12: [RCL] [+] {P} [ENTER]
Line 13: [RCL] {VM} [ENTER]
Line 14: [RCL] [-] {B} [ENTER]
Line 15: [×]
Line 16: [RCL] {R} [ENTER]
Line 17: [RCL] [×] {T} [ENTER]
Line 18: [-]
```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $P = 50$; $A = 3.592$; $B = 0.04267$; $R = 0.08314462618$; $T = 300$. Unknown: VM .

```
[SOLVER] {VDW} → 50 {P} → 3.592 {A} → 0.04267 {B} → 0.08314462618 {R} → 300 {T} →
0.05 {VM} → 1 {VM} → {VM}
```

VERIFICATION Expected result: $VM = 0.3696093626$ L/mol. Both sides evaluate to approximately 24.94338785 L·bar/mol.

What can go wrong

- Below the critical region the cubic may have three real roots.
- Do not mix $\text{Pa} \cdot \text{m}^3$ with $\text{bar} \cdot \text{L}$ constants.

WHY THIS WORKS The domain $VM > B$ is physical and also avoids the excluded-volume singularity.

Model note: The van der Waals model is qualitative near phase coexistence; real-fluid property work may require a modern equation of state.

10.3 Colebrook White friction factor

Find the Darcy friction factor for $Re = 100\,000$, pipe diameter 0.15 m and roughness 45 μm .

Variable	Meaning	Unit
F	Darcy friction factor	1
RE	Reynolds number	1
EPS	absolute roughness	m

Variable	Meaning	Unit
D	diameter	m
ISF	internal 1/√F	1

$$1/\sqrt{F} = -2 \log_{10}(\text{EPS}/(3.7D) + 2.51/(\text{RE} \sqrt{F}))$$

Residual: $f = 1/\sqrt{F} + 2 \log_{10}(\dots)$

Calculator mode: Any angle mode

Program listing

```

01 LBL "COLEBR"
02 MVAR "F"
03 MVAR "RE"
04 MVAR "EPS"
05 MVAR "D"
06 RCL "F"
07 SQRT
08 1/X
09 STO "ISF"
10 RCL "EPS"
11 3.7
12 RCL× "D"
13 ÷
14 2.51
15 RCL× "ISF"
16 RCL÷ "RE"
17 +
18 LOG
19 2
20 ×
21 RCL+ "ISF"
22 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {COLEBR} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {F} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {RE} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {EPS} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {D} [ENTER]
Line 06: [RCL] {F} [ENTER]
Line 07: [√x]
Line 08: [1/X]
Line 09: [STO] {ISF} [ENTER]
Line 10: [RCL] {EPS} [ENTER]
Line 11: [3] [.] [7]
Line 12: [RCL] [×] {D} [ENTER]
Line 13: [÷]
Line 14: [2] [.] [5] [1]
Line 15: [RCL] [×] {ISF} [ENTER]
Line 16: [RCL] [÷] {RE} [ENTER]
Line 17: [+]
Line 18: [LOG]
Line 19: [2]

```

Line 20: [×]

Line 21: [RCL] [+] (ISF) [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: RE = 100000; EPS = 45E-6; D = 0.15. Unknown: F.

[SOLVER] {COLEBR} → 100000 {RE} → 45E-6 {EPS} → 0.15 {D} → 0.005 {F} → 0.1 {F} → {F}

VERIFICATION Expected result: F = 0.019469127552. The independent residual magnitude is below 2E-14.

What can go wrong

- This is the Darcy factor, not the Fanning factor.
- The correlation applies to turbulent pipe flow; check Re first.
- $F \leq 0$ is outside the square-root domain.

WHY THIS WORKS The useful physical range is positive and narrow; 0.005 to 0.1 is a strong generic turbulent-flow bracket.

Model note: Colebrook and White's implicit correlation is conventionally written with base-10 logarithm.

10.4 Internal rate of return

Solve the NPV of six periodic cash flows for a single IRR. Cash flows are −1000, 200, 300, 400, 500 and 200.

Variable	Meaning	Unit
R	decimal rate per period	1/period
CF0	cash flow at t0	currency
CF1...5	later cash flows	currency
Q	internal 1+R	1

$$\text{NPV}(R) = \sum CF_t / (1+R)^t = 0$$

Residual: f = NPV(R)

Calculator mode: Any angle mode

Program listing

```

01 LBL "IRR"
02 MVAR "R"
03 MVAR "CF0"
04 MVAR "CF1"
05 MVAR "CF2"
06 MVAR "CF3"
07 MVAR "CF4"
08 MVAR "CF5"
09 RCL "R"
10 1
11 +
12 STO "Q"
13 RCL "CF5"
14 RCL÷ "Q"
15 RCL+ "CF4"
16 RCL÷ "Q"

```

```

17 RCL+ "CF3"
18 RCL÷ "Q"
19 RCL+ "CF2"
20 RCL÷ "Q"
21 RCL+ "CF1"
22 RCL÷ "Q"
23 RCL+ "CF0"
24 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {IRR} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {R} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {CF0} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {CF1} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {CF2} [ENTER]
Line 06: [PGM.FCN] [▲] {MVAR} {CF3} [ENTER]
Line 07: [PGM.FCN] [▲] {MVAR} {CF4} [ENTER]
Line 08: [PGM.FCN] [▲] {MVAR} {CF5} [ENTER]
Line 09: [RCL] {R} [ENTER]
Line 10: [1]
Line 11: [+]
Line 12: [ST0] {Q} [ENTER]
Line 13: [RCL] {CF5} [ENTER]
Line 14: [RCL] [÷] {Q} [ENTER]
Line 15: [RCL] [+] {CF4} [ENTER]
Line 16: [RCL] [÷] {Q} [ENTER]
Line 17: [RCL] [+] {CF3} [ENTER]
Line 18: [RCL] [÷] {Q} [ENTER]
Line 19: [RCL] [+] {CF2} [ENTER]
Line 20: [RCL] [÷] {Q} [ENTER]
Line 21: [RCL] [+] {CF1} [ENTER]
Line 22: [RCL] [÷] {Q} [ENTER]
Line 23: [RCL] [+] {CF0} [ENTER]

```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: CF0 = -1000; CF1 = 200; CF2 = 300; CF3 = 400; CF4 = 500; CF5 = 200. Unknown: R.

```

[SOLVER] {IRR} → -1000 {CF0} → 200 {CF1} → 300 {CF2} → 400 {CF3} → 500 {CF4} → 200
{CF5} → 0 {R} → 0.5 {R} → {R}

```

VERIFICATION Expected result: $R = 0.1691446452$, or 16.914465% per period. The independently recomputed NPV is below 10^{-12} currency units.

What can go wrong

- Rates $R \leq -1$ cross a denominator singularity.
- Non-conventional cash flows can have multiple IRRs.
- IRR alone does not measure scale or reinvestment reality.

WHY THIS WORKS Horner evaluation avoids repeated powers and reduces both program length and rounding exposure.

10.5 Catenary parameter from span and cable length

For supports at equal height, find catenary parameter a for a 42 m cable spanning 40 m horizontally.

Variable	Meaning	Unit
A	catenary parameter	m
S	horizontal span	m
L	cable length	m

$$L = 2A \sinh(S/(2A))$$

Residual: $f = 2A \sinh(S/(2A)) - L$

Calculator mode: Any angle mode

Program listing

```

01 LBL "CATEN"
02 MVAR "A"
03 MVAR "S"
04 MVAR "L"
05 RCL "S"
06 2
07 RCL× "A"
08 ÷
09 SINH
10 2
11 RCL× "A"
12 ×
13 RCL- "L"
14 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

Line 01: [PGM.FCN] {LBL} {CATEN} [ENTER]

Line 02: [PGM.FCN] [▲] {MVAR} {A} [ENTER]

Line 03: [PGM.FCN] [▲] {MVAR} {S} [ENTER]

Line 04: [PGM.FCN] [▲] {MVAR} {L} [ENTER]

Line 05: [RCL] {S} [ENTER]

Line 06: [2]

Line 07: [RCL] [×] {A} [ENTER]

Line 08: [÷]

Line 09: [HYP] [SIN]

Line 10: [2]

Line 11: [RCL] [×] {A} [ENTER]

Line 12: [×]

Line 13: [RCL] [-] {L} [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $S = 40$; $L = 42$. Unknown: A .

[SOLVER] {CATEN} → 40 {S} → 42 {L} → 1 {A} → 1000 {A} → {A}

VERIFICATION Expected result: $A = 36.7855852015$ m. $2A \sinh(40/(2A))$ evaluates to 42.000000000 m.

What can go wrong

- This ideal catenary assumes uniform load per horizontal or cable coordinate according to the chosen derivation; do not mix formulations.

WHY THIS WORKS Before solving, check $L \geq S$. If the cable is shorter than the span, the model has no real physical solution.

10.6 Radiation and convection equilibrium

Find the steady surface temperature of a 0.1 m^2 surface receiving 500 W and losing heat by convection and radiation.

Variable	Meaning	Unit
T	surface temperature	K
TA	ambient temperature	K
TS	radiative surroundings	K
H	convection coefficient	$\text{W}/(\text{m}^2 \cdot \text{K})$
EPS	emissivity	1
A	area	m^2
Q	input heat	W
SIG	Stefan-Boltzmann constant	$\text{W}/(\text{m}^2 \cdot \text{K}^4)$

$$H A(T - T_A) + \text{EPS SIG } A(T^4 - T_S^4) = Q$$

Residual: $f = \text{convective loss} + \text{radiative loss} - Q$

Calculator mode: Any angle mode

Program listing

```

01 LBL "RADHT"
02 MVAR "T"
03 MVAR "TA"
04 MVAR "TS"
05 MVAR "H"
06 MVAR "EPS"
07 MVAR "A"
08 MVAR "Q"
09 MVAR "SIG"
10 RCL "T"
11 RCL - "TA"
12 RCL× "H"
13 RCL× "A"
14 RCL "T"
15 X↑2
16 X↑2
17 RCL "TS"
18 X↑2
19 X↑2
20 -

```

```

21 RCL× "EPS"
22 RCL× "SIG"
23 RCL× "A"
24 +
25 RCL- "Q"
26 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} <RADHT> [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} <T> [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} <TA> [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} <TS> [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} <H> [ENTER]
Line 06: [PGM.FCN] [▲] {MVAR} <EPS> [ENTER]
Line 07: [PGM.FCN] [▲] {MVAR} <A> [ENTER]
Line 08: [PGM.FCN] [▲] {MVAR} <Q> [ENTER]
Line 09: [PGM.FCN] [▲] {MVAR} <SIG> [ENTER]
Line 10: [RCL] <T> [ENTER]
Line 11: [RCL] [-] <TA> [ENTER]
Line 12: [RCL] [×] <H> [ENTER]
Line 13: [RCL] [×] <A> [ENTER]
Line 14: [RCL] <T> [ENTER]
Line 15: [X+2]
Line 16: [X+2]
Line 17: [RCL] <TS> [ENTER]
Line 18: [X+2]
Line 19: [X+2]
Line 20: [-]
Line 21: [RCL] [×] <EPS> [ENTER]
Line 22: [RCL] [×] <SIG> [ENTER]
Line 23: [RCL] [×] <A> [ENTER]
Line 24: [+]
Line 25: [RCL] [-] <Q> [ENTER]

```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: TA = 298; TS = 295; H = 8; EPS = 0.85; A = 0.1; Q = 500; SIG = 5.670374419E-8.

Unknown: T.

```

[SOLVER] {RADHT} → 298 {TA} → 295 {TS} → 8 {H} → 0.85 {EPS} → 0.1 {A} → 500 {Q} →
5.670374419E-8 {SIG} → 298 {T} → 1000 {T} → {T}

```

VERIFICATION Expected result: T = 521.84257794 K (248.6926 °C). The two heat-loss mechanisms sum to 500.000000 W.

What can go wrong

- Using °C inside T^4 is catastrophic.
- H and emissivity are often uncertain and temperature-dependent.

WHY THIS WORKS Absolute temperature is mandatory in the T^4 radiation term. Celsius belongs only in reporting or in temperature differences.

10.7 Saturable elimination steady state

Solve a one-compartment steady-state balance with Michaelis-Menten elimination plus linear clearance.

Variable	Meaning	Unit
C	steady concentration	mg/L
RIN	input rate	mg/(L·h)
VMAX	maximum nonlinear elimination	mg/(L·h)
KM	half-saturation concentration	mg/L
KE	linear rate	1/h

$$RIN = VMAX \cdot C / (KM + C) + KE \cdot C$$

Residual: $f = RIN - \text{nonlinear loss} - \text{linear loss}$

Calculator mode: Any angle mode

Program listing

```

01 LBL "PKSS"
02 MVAR "C"
03 MVAR "RIN"
04 MVAR "VMAX"
05 MVAR "KM"
06 MVAR "KE"
07 RCL "VMAX"
08 RCL× "C"
09 RCL "KM"
10 RCL+ "C"
11 ÷
12 RCL "KE"
13 RCL× "C"
14 +
15 +/-
16 RCL+ "RIN"
17 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {PKSS} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {C} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {RIN} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {VMAX} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {KM} [ENTER]
Line 06: [PGM.FCN] [▲] {MVAR} {KE} [ENTER]
Line 07: [RCL] {VMAX} [ENTER]
Line 08: [RCL] [×] {C} [ENTER]
Line 09: [RCL] {KM} [ENTER]
Line 10: [RCL] [+] {C} [ENTER]
Line 11: [÷]
Line 12: [RCL] {KE} [ENTER]
Line 13: [RCL] [×] {C} [ENTER]

```

Line 14: [+]
 Line 15: [+/-]
 Line 16: [RCL] [+] (RIN) [ENTER]

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: RIN = 30; VMAX = 50; KM = 0.8; KE = 0.15. Unknown: C.

[SOLVER] {PKSS} → 30 {RIN} → 50 {VMAX} → 0.8 {KM} → 0.15 {KE} → 0 {C} → 20 {C} → {C}

VERIFICATION Expected result: C = 1.1824196113 mg/L. The two elimination terms sum to 30.000000 mg/(L·h).

What can go wrong

- Concentration must be non-negative.
- Units of RIN must match concentration-rate units.
- Parameter uncertainty can dominate numerical precision.

WHY THIS WORKS This is a modelling demonstration. Patient dosing requires validated parameters and clinical oversight.

Model note: The model combines a saturable term with first-order elimination; it is not a dosing recommendation.

10.8 Delivery cycle capacity with congestion

A stylised depot cycle takes $\text{TAU} = T_0 + Q/\text{RATE} + A Q^2$ hours for payload Q. Find payloads that deliver an average demand $D = Q/\text{TAU}$. The quadratic congestion term creates two mathematical operating points.

Variable	Meaning	Unit
Q	payload per cycle	unit
D	required average flow	unit/h
T0	fixed cycle time	h
RATE	handling rate	unit/h
A	congestion coefficient	h/unit ²

$$Q \div (T_0 + Q/\text{RATE} + A Q^2) = D$$

Residual: f = achieved average rate – D

Calculator mode: Any angle mode

Program listing

```
01 LBL "DELCAP"
02 MVAR "Q"
03 MVAR "D"
04 MVAR "T0"
05 MVAR "RATE"
06 MVAR "A"
07 RCL "Q"
08 RCL÷ "RATE"
09 RCL+ "T0"
```

```

10 RCL "Q"
11 X↑2
12 RCL× "A"
13 +
14 RCL "Q"
15 X↔Y
16 ÷
17 RCL- "D"
18 END

```

Program entry

Start: [GT0] [.] [.] [PRGM] (new program space, then Program-entry mode)

```

Line 01: [PGM.FCN] {LBL} {DELCAP} [ENTER]
Line 02: [PGM.FCN] [▲] {MVAR} {Q} [ENTER]
Line 03: [PGM.FCN] [▲] {MVAR} {D} [ENTER]
Line 04: [PGM.FCN] [▲] {MVAR} {T0} [ENTER]
Line 05: [PGM.FCN] [▲] {MVAR} {RATE} [ENTER]
Line 06: [PGM.FCN] [▲] {MVAR} {A} [ENTER]
Line 07: [RCL] {Q} [ENTER]
Line 08: [RCL] [÷] {RATE} [ENTER]
Line 09: [RCL] [+] {T0} [ENTER]
Line 10: [RCL] {Q} [ENTER]
Line 11: [X↑2]
Line 12: [RCL] [×] {A} [ENTER]
Line 13: [+]
Line 14: [RCL] {Q} [ENTER]
Line 15: [X↔Y]
Line 16: [÷]
Line 17: [RCL] [-] {D} [ENTER]

```

The new space already ends in .END.; finish: [PRGM]

Solve

Known data: $D = 10$; $T0 = 0.5$; $RATE = 20$; $A = 0.0005$. Unknown: Q .

```
[SOLVER] {DELCAP} → 10 {D} → 0.5 {T0} → 20 {RATE} → 0.0005 {A} → 1 {Q} → 50 {Q} → {Q}
```

VERIFICATION Expected result: $Q = 11.2701665379$ units; a second root is 88.7298334621. Both payloads yield an average rate of 10 units/h in the stated stylised model.

Store guesses 50 and 150 in Q to select the high-payload root 88.7298334621.

What can go wrong

- The quadratic congestion law is an explicit modelling assumption.
- Scan the domain because one bracket finds only one branch.
- Capacity, integer payloads and service constraints may rule out a root.

WHY THIS WORKS The smaller root is the low-congestion branch; the larger is a congestion-dominated branch. A mathematical root is not automatically the operationally preferred point.

PART VI

Highly Advanced Solver Techniques

Bracketing, transformations, automated SOLVE calls, reusable subroutines and robust model design.

11 Finding every relevant root

A single Solver run answers a local question: what root does this model and these guesses lead to? It does not enumerate all roots. Systematic work therefore separates reconnaissance from refinement.

11. Write the domain, including poles and physical limits.
12. Choose a coarse grid over the domain and evaluate the residual manually or with a scan program.
13. Record intervals with sign changes, but mark known discontinuities separately.
14. Search near suspected tangent roots by looking for a minimum of $|f|$ even when no sign change occurs.
15. Run SOLVE inside each clean interval.
16. Store each accepted root and continue outside a small exclusion neighborhood.

SOLVER TRAP An interval scan that only watches sign misses even-multiplicity roots and may falsely flag poles. Plot or inspect residual magnitude as well as sign.

11.1 One estimate two estimates and stored history

Storing one value into the unknown supplies one starting estimate. Storing a second value into the same unknown retains a pair for the Solver. The final result then replaces the named unknown, so a repeated solve starts from the recently found neighborhood unless you deliberately store new estimates.

- One estimate is convenient when the function is smooth and the root is nearby.
- Two opposite-sign estimates communicate a bracket.
- Two same-side estimates communicate scale and direction but no existence guarantee.
- After changing a parameter, refresh guesses if the root may have moved across a pole or branch boundary.

12 Transforming the unknown

When x must be positive over many orders of magnitude, solve for $u = \ln x$ and calculate $x = e^u$ inside the residual. When $0 < p < 1$, solve for an unconstrained u and map $p = 1/(1+e^{-u})$. These transformations remove invalid trial regions and often improve scaling.

Original constraint	Transform	Inverse used in the residual
$x > 0$	$u = \ln x$	$x = e^u$
$a < x < b$	u unconstrained	$x = a + (b-a)/(1+e^{-u})$
x spans decades	$u = \log_{10} x$	$x = 10^u$
x near a reference x_0	$u = (x-x_0)/s$	$x = x_0 + s u$

EXPERT TIP Transformations change which variable the Solver reports. Convert the final u back to x and verify the original, untransformed equation.

13 Piecewise models and protected domains

A residual program may branch by trial value, but every branch must return the same physical quantity in compatible units. Avoid discontinuous branch switches unless the model itself is discontinuous. If evaluation would be invalid, the most honest design is often to choose a safe transformed variable or a safe bracket rather than return an arbitrary large penalty.

- Document every boundary and whether the function is continuous there.
- Test the exact boundary, a value just below, and a value just above.
- Do not use a penalty that accidentally crosses zero.
- If a branch represents a different physical regime, verify that the selected root belongs to that regime.

14 Calling the Solver from another program

The HP-42S programming interface, retained by Free42 and the DM42, provides PGMSLV to select a Solver program and SOLVE to solve for a named variable. The return stack uses the same X/Y/Z/T structure, so a calling program can branch on the code in T.

```

01 LBL "AUTOKEP"
02 PGMSLV "KEPLER"
03 0.7
04 STO "M"
05 0.6
06 STO "ECC"
07 0.7
08 STO "E"
09 1.7
10 SOLVE "E"
11 GTO IND ST T
12 LBL 00
13 RTN
14 LBL 01
15 ... handle sign reversal ...
16 END

```

The first estimate is stored in the unknown. The second estimate is in X when SOLVE executes. `GTO IND ST T` dispatches to LBL 00 through LBL 04 according to the documented result code.

CAUTION Do not assume that a Solver program can safely invoke SOLVE recursively. This handbook documents top-level and programmatic SOLVE use, not recursive Solver nesting.

14.1 PGMMENU and PGMVAR

Current Free42 extensions exposed on the DM42 include PGMMENU, which presents a menu of global labels having MVAR declarations, and PGMVAR, which prints the MVAR variables associated with a label. They are useful when building a front end around a library of Solver models. They do not change the mathematics of SOLVE.

15 Reusable subroutines

A difficult residual may call a tested subroutine such as a normal CDF approximation. Keep the contract explicit: what must be in X or named variables, what is returned, which helper variables are overwritten, and which angle mode is required. A subroutine that leaves extra stack objects can make a correct-looking residual wrong.

Contract item	Question to answer
Input	Which register or named variables supply arguments?
Output	Is the scalar result returned in X?
Stack	Are Y, Z and T preserved or intentionally consumed?
State	Which flags, modes or helper variables change?
Domain	What input values cause an error?
Accuracy	What approximation and worst-case error apply?

16 Combining the Solver with other DM42 facilities

A residual can obtain a scalar from matrix calculations, statistics or numerical integration when that composition is genuinely part of the model. Keep the Solver's repeated-evaluation cost in mind. Precompute quantities that do not depend on the unknown and store them as named parameters.

- Matrices: use them to hold data or coefficients, but return one scalar residual.
- Statistics: fit parameters outside the Solver when possible; solve only the final implicit condition.
- Integration: an integral that depends on the unknown can be embedded, but repeated numerical integration may be slow and may compound error.
- Subroutines: isolate domain formulas and test them independently before placing them inside the iterative loop.

EXPERT TIP The fastest residual is often the one that moves invariant work outside the program body. Numerical elegance on a pocket calculator includes conserving repeated evaluations.

PART VII

Extreme Examples

Five reproducible applications that combine modelling, domain control, reusable code, multiple roots and sensitivity analysis.

These examples are longer because the model deserves explanation. Repetitive alpha-entry keystrokes are condensed, but every program line is shown and every unusual instruction is identified. Enter or encode the program, compare the stored listing line by line, then test the residual at a known point before using SOLVE.

17 Black Scholes implied volatility

A market price is inverted through a reusable normal CDF subroutine.

For a European call with no dividends, the Black-Scholes price is $C = S N(d_1) - K e^{-rT} N(d_2)$, where $d_1 = [\ln(S/K) + (r + \sigma^2/2)T]/(\sigma\sqrt{T})$ and $d_2 = d_1 - \sigma\sqrt{T}$. Market price C is known; volatility σ is the unknown.

Variable	Value	Meaning
S	100	spot price
K	100	strike price
R	0.05	continuously compounded annual risk-free rate
T	1	years to expiry
C	10.45	observed call price
SIG	unknown	annual volatility

$$f(\text{SIG}) = S N(d_1) - K e^{-RT} N(d_2) - C$$

The normal CDF subroutine uses the well-known five-coefficient rational approximation commonly attributed to Abramowitz and Stegun 26.2.17. Its absolute CDF error is about $7.5\text{E}-8$; the implied-volatility result should not be presented as more accurate than the approximation and market inputs justify.

Complete program

```

01 LBL "BSIV"
02 MVAR "S"
03 MVAR "K"
04 MVAR "R"
05 MVAR "T"
06 MVAR "C"
07 MVAR "SIG"
08 RCL "S"
09 RCL÷ "K"
10 LN
11 RCL "SIG"
12 X↑2
13 2
14 ÷
15 RCL+ "R"
16 RCL× "T"
17 +
18 RCL "SIG"
19 RCL "T"
20 SQRT

```

```

21  ×
22  ÷
23  STO "D1"
24  RCL "D1"
25  RCL "SIG"
26  RCL "T"
27  SQRT
28  ×
29  −
30  STO "D2"
31  RCL "D1"
32  XEQ "NCDF"
33  STO "N1"
34  RCL "D2"
35  XEQ "NCDF"
36  STO "N2"
37  RCL "S"
38  RCL× "N1"
39  RCL "K"
40  RCL "R"
41  RCL× "T"
42  +/-
43  E↑X
44  ×
45  RCL× "N2"
46  −
47  RCL- "C"
48  END
49  LBL "NCDF"
50  STO "XX"
51  ABS
52  .2316419
53  ×
54  1
55  +
56  1/X
57  STO "K0"
58  1.330274429
59  RCL× "K0"
60  1.821255978
61  −
62  RCL× "K0"
63  1.781477937
64  +
65  RCL× "K0"
66  .356563782
67  −
68  RCL× "K0"
69  .319381530
70  +
71  RCL× "K0"
72  STO "POLY"
73  RCL "XX"
74  X↑2
75  2
76  ÷
77  +/-
78  E↑X
79  2
80  PI
81  ×
82  SQRT
83  ÷
84  RCL× "POLY"
85  +/-
86  1
87  +
88  RCL "XX"
89  X<0?
90  GT0 00
91  RTN
92  LBL 00
93  +/-

```

```

94 1
95 +
96 RTN
97 END

```

Key entry guide

Start with [GTO] [.] [.] [PRGM]. Enter ordinary instructions with their physical keys or menu soft keys. Use [XEQ] only for the two `XEQ "NCDF"` lines, which call the CDF subroutine. For quoted parameters, type the name in the alpha menu and press ENTER. Direct numeric keys and arithmetic operators enter themselves. The other non-obvious lines are `X<0?`, which conditionally executes the next line, and local `LBL 00`/`GTO 00`, which reflect negative arguments.

Solve in ordinary real mode: [SOLVER] {BSIV}; store $S = 100$, $K = 100$, $R = 0.05$, $T = 1$ and $C = 10.45$; store guesses 0.05 and 0.5 in SIG; press {SIG}.

VERIFICATION The DM42 approximation gives $SIG = 0.199984659886$. A high-accuracy normal CDF gives 0.199984448011; the difference is $2.12e-07$.

Market call price	Implied σ	Interpretation
8.00	0.13377583	lower price implies lower volatility
10.45	0.19998445	base case
13.00	0.26753352	higher price implies higher volatility

- The call price must respect no-arbitrage bounds; otherwise no positive σ root exists.
- Very short expiries or deep in/out-of-the-money options can make implied volatility ill-conditioned.
- The formula assumes European exercise, constant volatility and rate, lognormal dynamics, frictionless markets and no dividends in this version.
- This is an educational numerical model, not investment advice.

18 High eccentricity orbital solve

A familiar equation becomes numerically demanding near $e = 1$ and small mean anomaly.

Reuse the KEPLER program from Part V with $M = 0.05$ rad and $ECC = 0.99$. The derivative $1 - e \cos E$ becomes small near $E = 0$, so a naive first estimate $E = M$ can be poor. The physical interval $0 \leq E \leq \pi$ is a safe bracket for $0 \leq M \leq \pi$.

Keystrokes: [SOLVER] {KEPLER} → 0.05 {M} → 0.99 {ECC} → 0 {E} → 3.141592654 {E} → {E}.

VERIFICATION $E = 0.645891456950$ rad. Substitution gives $E - 0.99 \sin(E) = 0.050000000000$ within independent double-precision rounding.

Eccentricity	Root E for $M = 0.05$ rad	Sensitivity
0.900	0.4027779387	moderate
0.990	0.6458914570	high
0.999	0.6716782961	very high near periapsis

EXPERT TIP For e close to 1, use a physical bracket rather than relying on one small estimate. The Solver's speed matters less than keeping every trial inside the intended orbital branch.

19 Diprotic acid speciation by charge balance

A multi-species chemical system is reduced to one scalar hydrogen-ion root.

For $\text{H}_2\text{A} \rightleftharpoons \text{H}^+ + \text{HA}^-$ and $\text{HA}^- \rightleftharpoons \text{H}^+ + \text{A}^{2-}$ with total analytical concentration CT and sodium counterion concentration NA , express all species as functions of $\text{H} = [\text{H}^+]$. With $\text{D} = \text{H}^2 + \text{K1 H} + \text{K1 K2}$, $[\text{HA}^-] = \text{CT K1 H/D}$ and $[\text{A}^{2-}] = \text{CT K1 K2/D}$. Charge balance gives $\text{H} + \text{NA} - \text{KW/H} - [\text{HA}^-] - 2[\text{A}^{2-}] = 0$.

Parameter	Value	Unit
CT	0.01	mol/L
NA	0.012	mol/L
K1	1E-3	mol/L
K2	1E-7	mol/L
KW	1E-14	(mol/L) ²

```

01 LBL "DIPH"
02 MVAR "H"
03 MVAR "CT"
04 MVAR "NA"
05 MVAR "K1"
06 MVAR "K2"
07 MVAR "KW"
08 RCL "H"
09 X↑2
10 RCL "K1"
11 RCL× "H"
12 +
13 RCL "K1"
14 RCL× "K2"
15 +
16 STO "DEN"
17 RCL "H"
18 RCL+ "NA"
19 RCL "KW"
20 RCL÷ "H"
21 -
22 RCL "CT"
23 RCL× "K1"
24 RCL× "H"
25 RCL÷ "DEN"
26 -
27 RCL "CT"
28 RCL× "K1"
29 RCL× "K2"
30 RCL÷ "DEN"
31 2
32 ×
33 -
34 END

```

Enter the program with the physical-key and menu method described earlier. Solve with a positive logarithmic bracket: store $1\text{E}-12$ and 0.1 in H, then press {H}. Never use $H = 0$ because KW/H is singular.

VERIFICATION $H = 3.98951720032\text{e-}07$ mol/L and $\text{pH} = 6.399079658$. The independently evaluated charge residual is below 10^{-12} mol/L.

NA mol/L	Calculated pH	Observation
0.009	3.999621	near first buffer region
0.012	6.399080	between dissociations
0.015	7.000130	near second region

- Concentrations must be positive and the H bracket must exclude zero.
- Activities are replaced by concentrations; this is an ideal-dilute approximation.
- Additional ions and electroneutrality terms must be added for a real formulation.

20 Multiple internal rates of return

Two mathematically correct roots reveal why a Solver answer is not automatically a financial decision.

The cash flows -100 , $+230$ and -132 have two sign changes, so multiple positive IRRs are possible. The residual is $\text{NPV}(R) = -100 + 230/(1+R) - 132/(1+R)^2$.

```

01 LBL "MIRR"
02 MVAR "R"
03 RCL "R"
04 1
05 +
06 STO "Q"
07 230
08 RCL÷ "Q"
09 132
10 RCL "Q"
11 X+2
12 ÷
13 -
14 100
15 -
16 END

```

First solve with 0.01 and 0.15 stored in R. Then overwrite the guesses with 0.15 and 0.50 and solve again.

VERIFICATION The two roots are $R = 0.100000000000$ and $R = 0.200000000000$, exactly 10% and 20% within rounding. Both make NPV zero.

SOLVER TRAP Both IRRs are mathematically real; neither is automatically the economically correct summary. Examine the cash-flow pattern, financing assumptions and NPV at an externally chosen discount rate.

21 Electrothermal cable equilibrium

Temperature-dependent electrical heating is balanced against convection and radiation.

For one metre of round conductor, use $I^2 R_{20} [1 + \alpha(T - T_0)] = hA(T - T_A) + \epsilon \sigma A(T^4 - T_S^4)$, with $A = \pi D$. The unknown T appears linearly and to the fourth power, and resistance also changes with T .

Quantity	Value	Unit
I	40	A
R20	0.01	Ω/m
α	0.00393	1/K
T0	293.15	K
h	10	$\text{W}/(\text{m}^2 \cdot \text{K})$
D	0.01	m
ϵ	0.8	1
TA = TS	298.15	K

```

01 LBL "CABLE"
02 MVAR "T"
03 MVAR "I"
04 MVAR "R20"
05 MVAR "ALPHA"
06 MVAR "T0"
07 MVAR "H"
08 MVAR "D"
09 MVAR "EPS"
10 MVAR "SIG"
11 MVAR "TA"
12 MVAR "TS"
13 PI
14 RCL× "D"
15 STO "AREA"
16 RCL "T"
17 RCL - "T0"
18 RCL× "ALPHA"
19 1
20 +
21 RCL× "R20"
22 RCL "I"
23 X↑2
24 ×
25 RCL "T"
26 RCL - "TA"
27 RCL× "H"
28 RCL× "AREA"
29 -
30 RCL "T"
31 X↑2
32 X↑2
33 RCL "TS"
34 X↑2
35 X↑2
36 -
37 RCL× "EPS"
38 RCL× "SIG"
39 RCL× "AREA"
40 -
41 END

```

Store SIG = 5.670374419E-8. Bracket T between ambient 298.15 K and 1000 K. The residual is electrical heating minus convective and radiative loss.

VERIFICATION T = 335.788057020 K = 62.638057 °C. Heating and total losses agree to better than 10^{-12} W/m in the independent calculation.

Current A	Equilibrium K	Equilibrium °C
30	318.7151	45.5651
40	335.7881	62.6381
50	358.8808	85.7308

- The convection coefficient is uncertain and installation-specific.
- Real ampacity work must include insulation, geometry, wind, solar load, contacts and safety standards.
- The example demonstrates coupled nonlinear balance, not a cable-rating recommendation.

APPENDICES

Quick Reference and Indexes

Procedures, messages, glyphs, constants, troubleshooting, glossary and cross-indexes.

Appendix A Solver in 60 seconds

17. Write the equation and choose one unknown.
18. Move all terms to one side so the program returns $f(x) = 0$.
19. Check units, domain, angle mode, poles and likely root range.
20. Select [SOLVER] and the global label of the model.
21. Store every known MVAR by typing the value and pressing its soft key.
22. Store one estimate, or two estimates that preferably bracket a continuous root, into the unknown.
23. Press the unknown soft key with no number pending.
24. Read X as the candidate, Y as the previous estimate, Z as $f(X)$, and T as result code 0–4.
25. Substitute X into the original equation and reject invalid physical roots.

KEY IDEA Before reaching for SOLVE, look at the equation. The calculator can search quickly; it cannot decide what the model means.

Appendix B Standard Solver program skeleton

```

01  LBL "MODEL"
02  MVAR "X"
03  MVAR "P1"
04  MVAR "P2"
05  ... recall values and calculate the residual ...
06  END

```

Keep all MVAR declarations together immediately after the global label. Use helper variables inside the calculation without declaring them as MVAR if the user should not edit them.

Appendix C Keystroke notation legend

Form	Meaning
[PRGM]	Named physical or shifted calculator function
{X}	Current soft-key label on the LCD
(MVAR)	Literal alpha characters typed, followed by ENTER
value {A}	Store value into variable A from a variable menu
{A}	With no pending number, select A as the unknown
→	Proceed to the next key group; not a calculator key unless enclosed in brackets

Appendix D Solver messages and actions

Message	T	Likely cause	What to do
none / candidate	0	Root candidate found	Inspect Z and substitute
Sign Reversal	1	Divergent opposite signs near a pole	Check continuity and neighboring values
Extremum	2	Minimum or maximum of f	Inspect Z; consider tangent root or no root
Bad Guess(es)	3	Initial point outside domain	Move both guesses inside the valid domain
Constant?	4	Flat, unscaled, or unknown not used	Test sensitivity and program references

Appendix E Initial guess checklist

- Is the interval inside the mathematical domain?
- Is it inside the physical domain?
- Is there a pole or branch boundary between the estimates?
- Do the endpoint residuals have opposite signs? If not, is a tangent root expected?
- Could another root lie in the same interval?
- Are the estimates expressed in the current angle and unit convention?
- Would a transformed unknown provide a safer interval?

Appendix F Common equation to residual transformations

Original form	Residual form
$A = B$	$A - B$
$A/B = C$	$A/B - C$, with $B \neq 0$
$A = e^x$	$e^x - A$
$A = \ln x$	$\ln x - A$, with $x > 0$
$x = g(x)$	$g(x) - x$
$\sum \text{cash flow}/(1+r)^t = 0$	NPV(r) directly
vector balance	Reduce to one scalar only when the model truly has one degree of freedom

Appendix G Frequently used DM42 functions

Function	Solver use
LBL	Defines the globally selectable model
MVAR	Declares a Solver variable-menu item
SOLVER	Opens the top-level Solver application
PGMSLV	Selects a Solver model from a program
SOLVE	Solves for a named variable from a program
PGMMENU	Shows global labels having MVAR declarations
PGMVAR	Prints the MVAR set for a label
RCL, RCL+, RCL−, RCL×, RCL÷	Reads named values and combines them
STO	Stores helpers or known values
LN, LOG, E↑X, 10↑X	Exponential and logarithmic residuals
SIN, COS, TAN	Angle-mode-sensitive models
LN1+X, E↑X−1	Stable evaluation near zero

Appendix H DM42 glyph reference

Preferred glyph	Meaning in listings	Avoid in main listings
×	multiply	*
÷	divide	/
√	square root	sqrt
Σ	summation	SUM when the actual instruction is Σ
∫	integral	INT as a visual substitute
π	pi	PI in prose
μ	micro	u when a true glyph is available
°	degree	deg

Preferred glyph	Meaning in listings	Avoid in main listings
$\leq \geq \neq$	comparisons	$<= >= !=$
$\rightarrow \leftarrow \uparrow \downarrow$	arrows and conversions	$-> <- ^\wedge \vee$
E	base-10 exponent marker	E in calculator-facing numerals
$R\uparrow R\downarrow$	roll stack	$R^\wedge$ RDN in the primary listing
$STO\times STO\div$	storage arithmetic	$STO* STO/$
$\rightarrow POL \rightarrow REC$	coordinate conversion	R-P P-R

Appendix I SwissMicros Encoder Decoder mnemonics

The preferred handbook notation is the actual calculator glyph. The following case-sensitive text codes are useful only when preparing source text for the official Encoder/Decoder.

Glyph or instruction	Encoder text	Alternative accepted form
$\div$	<code>\:-</code>	/ in an instruction
$\times$	<code>\x</code>	x or * in an instruction
$\sqrt{}$	<code>\w/</code>	—
$\int$	<code>\S</code>	—
Σ	<code>\GS</code>	SUM mnemonics for related instructions
π	<code>\PI</code>	—
$\leq$	<code>\<=</code>	$X<=0?$ for the instruction
$\geq$	<code>\>=</code>	$X>=0?$
$\neq$	<code>\#</code>	$X\#0?$
$\rightarrow$	<code>\-></code>	$\->POL$ etc.
$\leftarrow$	<code>\<-</code>	—
μ	<code>\m</code>	—
$^\circ$	<code>\o</code>	—
E	<code>\E</code>	—

Glyph or instruction	Encoder text	Alternative accepted form
R↑	R^	—
R↓	R\w or Rv	RDN
STO×	STO*	ST*
STO÷	STO/	ST/

Appendix J Unit and conversion reminders

Quantity	Reminder
Angles	State DEG or RAD before every trigonometric solve
Temperature	Use kelvin in absolute-temperature powers and gas laws
Percent	Convert 8% to 0.08 unless the model explicitly divides by 100
Micro	$\mu = 10^{-6}$; enter 45 μm as 45E-6 m
Pressure-volume	1 bar·L = 100 J; keep the gas constant in the same system
Finance	Match rate period to cash-flow period
Friction factor	Distinguish Darcy from Fanning

Appendix K Selected constants used

Constant	Value used	Unit or note
standard gravity g_0	9.80665	m/s ² , exact conventional value
molar gas constant R	8.31446261815324	J/(mol·K), 2019 SI exact from k and N _A
R in bar·L	0.08314462618	L·bar/(mol·K), rounded
Stefan-Boltzmann σ	5.670374419E-8	W/(m ² ·K ⁴), exact from 2019 SI constants
water specific heat	4184	J/(kg·K), illustrative near room temperature

Constant	Value used	Unit or note
air density	1.225	kg/m ³ , illustrative sea-level value

Appendix L Troubleshooting checklist

Symptom	Check first	Then
No model in SOLVER	Global LBL and contiguous MVAR lines	Check label uniqueness and END
Wrong root	Guesses and physical interval	Scan for other roots
Bad Guess(es)	Log, root and denominator domains	Transform the unknown
Sign Reversal	Known poles	Evaluate just left and right
Extremum	Magnitude of Z	Decide whether tangent root or no zero
Constant?	Does the program actually RCL the unknown?	Rescale the residual
Plausible but wrong trig answer	DEG/RAD annunciator	Re-enter all angular values
Exponent off by powers of ten	E entry and unit prefix	Write the conversion explicitly
Program and equation disagree	RPN stack order	Single-step and record X/Y/Z/T

Appendix M Worked examples by discipline

Discipline	Examples
Astrophysics	Kepler's equation
Biology	Logistic population time; Michaelis Menten substrate concentration
Business	Percentage increase
Chemistry	Ideal gas pressure; Weak acid equilibrium; Van der Waals molar volume
Classical physics	Force mass and acceleration; Distance at constant speed; Projectile range and two launch angles; Quadratic drag terminal speed

Discipline	Examples
Electronics	Ohm's law; Electrical power; RC charging time; Diode and resistor operating point
Engineering	Mass density and volume
Finance	Simple interest; Compound growth rate; Loan interest rate; Internal rate of return
Fluid engineering	Colebrook White friction factor
Game theory	Quantal response equilibrium
Logistics	Delivery cycle capacity with congestion
Mathematics	A first zero x plus 3 equals 7; One reusable model A plus B equals C
Mechanical engineering	Catenary parameter from span and cable length
Operations	Break even quantity
Pharmacokinetics	Saturable elimination steady state
Thermal engineering	Sensible heat; Radiation and convection equilibrium
Astrophysics	Kepler's equation; high-eccentricity orbital solve
Chemistry	Weak acid equilibrium; van der Waals molar volume; diprotic acid speciation
Finance	Simple interest; compound growth; loan rate; IRR; multiple IRRs; implied volatility
Thermal and electrical design	Radiation-convection balance; electrothermal cable equilibrium

Appendix N Worked examples by mathematical technique

Technique	Examples
Linear	$x + 3 = 7$; $A + B = C$; force; distance; Ohm's law; power
Polynomial or power	terminal velocity; projectile range; radiation; cable equilibrium
Exponential	RC charging; diode; logistic growth; compound interest
Logarithmic	Colebrook-White; transformed positive variables

Technique	Examples
Trigonometric	projectile; Kepler's equation
Hyperbolic	catenary
Fixed point	quantal response equilibrium
Implicit balance	van der Waals; chemical speciation; pharmacokinetics
Multiple roots	projectile angles; delivery capacity; multiple IRRs
Discontinuity or pole	weak-acid boundary; Colebrook domain; diagnostic pole example
Reusable subroutine	Black-Scholes implied volatility with NCDF

Glossary

Term	Meaning
bracket	Two estimates delimiting an interval, ideally with opposite residual signs and no discontinuity
conditioning	Sensitivity of the desired result to small changes in data or arithmetic
domain	Set of trial values for which the residual is mathematically defined
MVAR	Instruction declaring a variable in a Solver model's menu
residual	Scalar mismatch $f(x)$ whose desired value is zero
root or zero	Value of x for which $f(x) = 0$
scaling	Choice of variable and residual magnitudes for numerically useful evaluation
tangent root	Even-multiplicity root where the graph touches zero without changing sign
physical root	Mathematical root that also satisfies units, bounds and modelling assumptions

References

- SwissMicros, DM42 User Manual v3.42, 24 September 2026.
https://technical.swissmicros.com/dm42/doc/dm42_user_manual.html

2. SwissMicros, DM42 Quick Reference Guide, current online help.
<https://technical.swissmicros.com/dm42/fat/HELP/dm42help.htm>
3. SwissMicros, DM42 Functions documentation.
<https://technical.swissmicros.com/dm42/fat/HELP/dm42functs.htm>
4. SwissMicros, DM42 Encoder/Decoder instructions and glyph mappings.
<https://dm42decoder.swissmicros.com/>
5. Hewlett-Packard, HP-42S Owner's Manual, Chapter 12 The Solver, 1988.
<https://literature.hpcalc.org/community/hp42s-om-en.pdf>
6. Hewlett-Packard, HP-42S Programming Examples and Techniques, Chapter 3 The Solver, 1988.
<https://literature.hpcalc.org/community/hp42s-prog-en.pdf>
7. Hewlett-Packard, HP-42S Quick Reference Guide, 1988. <https://literature.hpcalc.org/community/hp42s-qrg-en.pdf>
8. Thomas Okken, Free42 documentation and extensions. <https://thomasokken.com/free42/>
9. Thomas Okken, Free42 project history, including SOLVE changes in releases 3.0.12 and 3.0.16.
<https://thomasokken.com/free42/history.html>
10. Colebrook, C. F., Turbulent Flow in Pipes, with Particular Reference to the Transition Region Between the Smooth and Rough Pipe Laws, Journal of the Institution of Civil Engineers, 1939.
<https://doi.org/10.1680/ijoti.1939.13150>
11. Black, F. and Scholes, M., The Pricing of Options and Corporate Liabilities, Journal of Political Economy 81(3), 1973. <https://www.jstor.org/stable/1831029>
12. Abramowitz, M. and Stegun, I. A., Handbook of Mathematical Functions, formula 26.2.17, 1964.
<https://personal.math.ubc.ca/~cbm/aands/>
13. NIST, CODATA and SI constants resources. <https://physics.nist.gov/cuu/Constants/>
14. IUPAC Gold Book, definitions and terminology for chemical equilibrium and pH. <https://goldbook.iupac.org/>

CAUTION Specialised equations are included as educational models. Apply domain-specific standards, current data, uncertainty analysis and professional judgement in consequential work.